CONTEXT-AWARE SINHALA POST-OCR ERROR CORRECTION FOR ASSISTIVE DOCUMENT READING

H.P. Ishara Madhusanka

IT22259134

Project ID: R26-DS-002

Supervisor: Ms. Lakmini Abeywardhana
Co-supervisor: Ms. Adya Dissanayaka

Report submitted in partial fulfilment of the requirements for the

Bachelor of Science (Honours) in Information Technology

Specializing in Data Science

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

DECLARATION

I declare that this is my own work and this dissertation does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or institute of higher learning, and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

I also hereby grant to the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute my dissertation, in whole or in part, in print, electronic or other medium. I retain the right to use this content in whole or part in future works, such as articles or books.

Student name: Madhusanka HPI

Student ID: IT22259134

Signature: : ..... Date: : ..... 2026/10/10.....

The above candidate has carried out research for the undergraduate dissertation under my supervision.

Signature of the supervisor: .....

Date: .....

Signature of the co-supervisor: .....

Date: .....

ABSTRACT

Printed Sinhala is inaccessible to blind and low-vision readers in Sri Lanka. Mainstream on-device text recognition does not support the script and the cloud service classifies it as experimental, leaving an open source engine as the only practical route. On hand held photographs of Sinhala newspaper pages it produces a character error rate of 0.1197 and a word error rate of 0.3358: roughly one spoken word in three is wrong, and a listener cannot detect it.

This reports a context-aware post-OCR correction component for Sinhala. A corpus of 232 photographed newspaper pages was transcribed by hand, and a frequency weighted taxonomy of 192 character confusions measured from it drove the generation of a synthetic training corpus, since no annotated parallel corpus of recognised and correct Sinhala exists. Five systems were compared on 217 sentence pairs split at photograph level and locked before any model was trained.

A fine-tuned mT5-small corrector reduced character error to 0.0757 and word error to 0.1640, reductions of 36.7 and 51.2 percent, 95 percent interval [-0.0615, -0.0199]. Dictionary and n-gram baselines gave no improvement and a degradation. The proposed detector-gated architecture, judged against a pre-registered win condition, underperformed the full-sequence control at 0.1160 and did not approach it at any of nine detector thresholds; a reachability ratio of 0.34 locates the cause in the corrector, not the detector. Gating does reduce damage to already-correct words fivefold.

Increasing real training material sixfold moved the result by 0.0006, placing the remaining ceiling at the optics, so capture was respecified as a target glyph height and enforced on a handset by a blur-invariant estimator. In a listening study, six participants recovered 60.0 percent of words from uncorrected speech and 79.2 percent after correction, a paired difference of +0.1917, 95 percent interval [+0.0958, +0.2882].

Keywords: post-OCR correction, Sinhala, low-resource NLP, assistive technology, sequence-to-sequence models

ACKNOWLEDGEMENTS

I wish to thank my supervisor, Ms. Lakmini Abeywardhana, and my co-supervisor, Ms. Adya Dissanayaka, for their guidance throughout this research. I thank my fellow group members for their collaboration on the integrated system, and the Department of Computer Science of the Sri Lanka Institute of Information Technology for the facilities and support that made this work possible.

TABLE OF CONTENT

LIST OF FIGURES xi

LIST OF TABLES xii

1 INTRODUCTION 1

1.1 Background 1

1.2 Problem Statement 1

1.3 Research Gap 2

1.4 Research Question 2

1.5 Aim and Objectives 2

1.6 Scope and Delimitations 3

2.1 Review Method and Scope 5

2.2 Assistive Reading for Blind and Low-Vision Users 5

2.2.1 The population and what it asks for 5

2.2.2 What existing reading aids achieve, and where they stop 6

2.2.3 Photographing a document without sight 6

2.2.4 The spoken channel, and what errors cost in it 6

2.3 Optical Character Recognition for Sinhala 7

2.3.1 The script and the difficulty it creates 7

2.3.2 Recognition engines and their reported accuracy 7

2.3.3 The gap between benchmark and page 8

2.4 Post-OCR Error Correction 8

2.4.1 The problem and its standard decomposition 8

2.4.2 Lexical and statistical approaches 9

2.4.3 Neural sequence-to-sequence correction 9

2.4.4 Synthetic training data 10

2.4.5 Detection-then-correction architectures 10

2.4.6 Large language models as correctors 11

2.5 Post-OCR Correction for Low-Resource and Indic Scripts 12
2.6 Sinhala Language Resources 13
2.7 Image Quality, Capture and Multi-Frame Methods 13
2.7.1 How much resolution recognition needs 13
2.8 Evaluation Practice and its Pitfalls 14
3.1 Research Design 15
3.2 Data and Study Materials 16
3.2.1 Source material and capture 16
3.2.2 Optical character recognition 17
3.2.3 Alignment and sentence-pair extraction 17
3.2.4 The locked evaluation split 18
3.2.5 Synthetic training corpus 19
3.3 Study Procedure 20
3.3.1 Construction of the error taxonomy 20
3.3.2 Generation of synthetic training pairs 21
3.3.3 Systems compared 22
3.3.4 Selection of the detector operating threshold 23
3.3.5 Training configuration 24
3.3.6 Inference configuration 24
3.4 Measures and Evaluation Metrics 25
3.4.1 Character and word error rate 25
3.4.2 Over-correction 25
3.4.3 Per-category error analysis 26
3.4.4 Reachability of the gated architecture 26
3.4.5 Two measurement conventions 27
3.5 Data Analysis 27
3.6 Ethics, Privacy, Security and Responsible Use of Automated Tools 28

3.7 Reproducibility and Research Artefacts 29
4 PRODUCT AND SOLUTION DESIGN 30
4.1 Requirements Specification 30
4.1.1 How these requirements were derived 30
4.1.2 Functional requirements 30
4.1.3 Non-functional requirements 32
4.1.4 Test case scenarios 33
4.2 System and Component Architecture 34
4.2.1 Position of the component within the system 34
4.2.2 Internal structure of the correction path 34
4.2.3 The data contract 36
4.3 Design Alternatives Considered 36
4.3.1 Four processes over HTTP against one shared interpreter 36
4.3.2 Specifying capture as a glyph height against specifying a camera 37
4.3.3 Which quantity the handset measures 37
4.3.4 The article detector against column projection 37
4.4 Social, Ethical and Security Aspects of the Design 38
5 IMPLEMENTATION, INTEGRATION AND TESTING 40
5.1 Development Environment and Technology Stack 40
5.1.1 Hardware and operating environment 40
5.1.2 Software stack and the reason the versions are pinned 41
5.1.3 Repository organisation and version control 41
5.2 Implementation of the Reading Path 42
5.2.1 Layer structure 42
5.2.2 Capture and frame selection 42
5.2.3 Region isolation 44
5.2.4 Recognition and correction 44

5.2.5 Batched decoding 45
5.2.6 The readability gate 46
5.2.7 Session state and follow-up commands 46
5.2.8 Configuration as a single source of measured constants 47
5.3 Integration with the Other Components 47
5.3.1 Process isolation over HTTP 47
5.3.2 The contract and the API surface 48
5.3.3 What leaves this component 48
5.4 Verification and Software Testing 49
5.4.1 Testing strategy 49
5.4.2 Composition of the suite 49
5.4.3 Current outcome, including the two failures 50
5.4.4 Model verification 50
5.5 Deployment and Operation 50
5.5.1 Running the system 50
5.5.2 Secrets and repository hygiene 51
6.1 Research Results 52
6.1.1 How much error there is to correct 52
6.1.2 Comparison of the five systems 52
6.1.3 Statistical significance 53
6.1.4 Why gating fails, and that it is not a tuning problem 54
6.1.5 Over-correction: the one dimension on which gating wins 55
6.1.6 Where the improvement comes from 56
6.1.7 Data scaling and the position of the ceiling 57
6.2 Product and Technical Results 58
6.2.1 The capture-resolution sweep 58
6.2.2 Multi-frame consensus 58

6.2.3 End-to-end measurement, and a disagreement between two protocols ... 59
6.2.4 Validation of the capture application ... 60
6.3 User and Stakeholder Validation ... 61
6.3.1 What this validation is for ... 61
6.3.2 Design of the listening study ... 61
6.3.3 Results ... 62
6.3.4 Interpretation ... 64
6.3.5 Limitations of this validation ... 64
6.4 Integration Results ... 65
6.4.1 Which region-isolation method is used, and why ... 65
6.4.2 Headline attachment ... 65
7 DISCUSSION ... 67
7.1 Interpretation of the Principal Result ... 67
7.2 Why the Proposed Architecture Underperformed ... 68
7.3 Comparison with Related Work ... 69
7.3.1 Against synthetic-data sequence-to-sequence post-OCR correction ... 69
7.3.2 Against existing Sinhala correction work ... 70
7.4 Contributions ... 71
7.5 Limitations and Threats to Validity ... 72
7.5.1 Threats to internal validity ... 72
7.5.2 Threats to external validity ... 73
7.5.3 Threats to the claim of usefulness ... 73
8 COMMERCIALISATION AND SUSTAINABILITY ... 74
8.1 Value Proposition and Target Users ... 74
8.1.1 The problem in commercial terms ... 74
8.1.2 Who the users are ... 74
8.2 Market and Competitive Position ... 75

8.2.1 Size of the need, from published statistics 75
8.2.2 What a Sinhala reader can use today 76
8.3 Sustainability Model 77
8.3.1 Why a conventional commercial model does not fit 77
8.3.2 The proposed model 77
8.3.3 Cost structure 77
8.3.4 Route to the user 78
8.4 Scalability, Intellectual Property and Compliance 78
8.4.1 Technical scalability 78
8.4.2 Scaling to other languages and domains 79
8.4.3 Compliance and responsible deployment 79
9 CONCLUSION AND FUTURE DIRECTIONS 80
9.1 Conclusion 80
9.2 Achievement of the Objectives 81
9.3 Future Directions 82
9.3.1 Immediate: close the gaps this work leaves open 82
9.3.2 Near term: test the boundaries of the result 83
9.3.3 Longer term: the directions the findings point to 83
REFERENCES 84
APPENDIX A: RELEASED ARTEFACTS AND REPRODUCTION 87
APPENDIX B: DISCLOSURE OF GENERATIVE AI USE 88

LIST OF FIGURES

Figure 3.1: Corpus construction and the locked page-disjoint split 19

Figure 4.1: The reading path, with the layers owned by this component shaded34

Figure 6.1: Character and word error rate for the five systems compared. The dashed line marks uncorrected recognition. 51

Figure 6.2: Gated character error rate against detector recall across nine thresholds. The architecture does not approach the full-sequence control at any operating point. 53

Figure 6.3: Proportion of error operations repaired, by error category 55

Figure 6.4: Character error rate against the downscale factor applied before recognition (item-level, normalised, uncorrected) 57

Figure 6.5: Left, word accuracy per participant in each condition, with the mean in bold. Right, the gain in word accuracy for each of the ten items. 61

LIST OF TABLES

Table 3.1: Classification of aligned sentence pairs 18
Table 3.2: The locked, page-disjoint evaluation split 19
Table 3.3: Composition of the scraped clean-text corpus 20
Table 3.4: Most frequent measured confusions, with occurrence counts.....21
Table 3.5: Training configuration of the three models.....24
Table 4.1: Functional requirements, acceptance criteria and verification.....30
Table 4.2: Non-functional requirements, acceptance criteria and status.....31
Table 5.1: Development and measurement environment.....39
Table 5.2: Batch size verification on the locked test set (n = 217) 44
Table 5.3: HTTP endpoints exposed by the component 46
Table 6.1: System comparison on the locked test 51
Table 6.2: Paired bootstrap % CI on the CER difference 52
Table 6.3: Over-correction on the locked test set.....54
Table 6.4: Fix rate by error group 55
Table 6.5: End-to-end, unaligned item-level protocol (n = 98 items).....58
Table 6.6: End-to-end, aligned sentence-level protocol (n = 217).....58
Table 6.7: Listening study outcome (6 participants, 60 responses) 61
Table 7.1: Relative CER reduction synthetic-data post-OCR correction.....68
Table 8.1: Published figures describing visual difficulty in Sri Lanka 73
Table 8.2: Alternatives available to a Sinhala reader who cannot read print..74
Table 9.1: Registered objectives and outcomes 79

LIST OF ABBREVIATIONS

Abbreviation	Meaning
API	Application Programming Interface
BERT	Bidirectional Encoder Representations from Transformers
CER	Character Error Rate
CI	Confidence Interval
CUDA	Compute Unified Device Architecture
EXIF	Exchangeable Image File Format
FPR	False-Positive Rate
FR	Functional Requirement
GPU	Graphics Processing Unit
HTTP	Hypertext Transfer Protocol
ICDAR	International Conference on Document Analysis and Recognition
JSON	JavaScript Object Notation
LO	Learning Outcome
mT5	Multilingual Text-to-Text Transfer Transformer
NFC	Normalization Form Canonical Composition
NFR	Non-Functional Requirement
OCR	Optical Character Recognition
PSM	Page Segmentation Mode
SDG	Sustainable Development Goal
SLIIT	Sri Lanka Institute of Information Technology
TAF	Topic Assessment Form
TTL	Time To Live
TTS	Text-to-Speech
WER	Word Error Rate
YOLO	You Only Look Once (object detector family)

1 INTRODUCTION

1.1 Background

Reading printed text is the capability blind and low-vision people name first when asked what they want from assistive technology [1]. The technology to provide it has existed for years: photograph the page, recognise the characters, speak the result. For a reader of English, Hindi or Japanese it is an ordinary feature of a telephone. For a reader of Sinhala it does not work in the ordinary case, and works poorly in the best case.

Sinhala is the first language of roughly three quarters of the population of Sri Lanka. It is written in an abugida of its own, occupying the Unicode block U+0D80 to U+0DFF, in which each consonant carries an inherent vowel modified by dependent vowel signs attached above, below, before or after the base character. The script is not supported by the dominant on-device text recognition library for Android [2], and the corresponding cloud service classifies it as experimental [3]. Dedicated commercial reading devices do not offer the language at all [4].

What remains is the open-source path: a general-purpose recognition engine with Sinhala training data, feeding a speech synthesiser. It produces Sinhala text and speaks it. Measured on hand held photographs of Sinhala newspaper pages it produces a character error rate of 0.1197 and a word error rate of 0.3358. Approximately one word in three is wrong.

1.2 Problem Statement

Recognised Sinhala newspaper text, obtained from photographs of the kind a blind user would produce, contains errors at a rate that prevents a listener from following it. The errors are dominated by confusions between visually similar characters at newsprint scale, principally dependent vowel signs and consonants, and a large proportion of them produce another valid Sinhala word rather than a non-word. Dictionary and spelling correction cannot detect such errors, because nothing about the resulting word is wrong in isolation; the evidence lies in the surrounding sentence. For morphologically rich Indic languages this is not an edge case: more than a third of single-character substitutions produce another real word, against 5 percent for English [5].

The problem is therefore to repair recognised Sinhala using the context of the sentence, with no annotated corpus of recognised and correct Sinhala text in existence to learn from, and to establish honestly whether the repair is large enough to matter to the person listening.

1.3 Research Gap

Chapter 2 develops the gap against the reviewed literature. In summary, four things are absent.

- There is no context-aware, transformer-based post-OCR corrector for Sinhala. The prior Sinhala correction work is rule and dictionary based [6][7].
- There is no frequency-weighted Sinhala recognition error taxonomy measured from real photographs rather than from synthetic corruption or clean scans.
- There is no locked, page-disjoint benchmark of real photographed Sinhala on which competing correction methods could be compared. Favourable published Sinhala recognition figures are measured on synthetic material, and the same engines measure error rates an order of magnitude higher on authentic degraded pages [8].
- There is no quantitative evaluation of whether a detector-gated architecture, in which errors are first flagged and only the flagged spans corrected, is the right design for an abugida in which the dominant errors are sub-character marks. Where the architecture has been isolated and measured, on European languages, it has been reported to help [9], and that evidence is thin.

1.4 Research Question

Can context-aware post-OCR correction reduce the error rate of recognised Sinhala newspaper text, photographed under the conditions a blind user would actually produce, by enough to change what a listener hears and is a detector-gated architecture the right way to do it?

The second half of the question carries equal weight deliberately. The architecture was the registered novelty of this project, and the study was designed so that it could fail.

1.5 Aim and Objectives

The aim of this research is to design, build and honestly evaluate a context-aware post-OCR error correction component for Sinhala, deployed inside an assistive reading

system for blind and low-vision readers, and to establish the conditions under which it works and the conditions under which it does not.

The objectives are as follows.

- O1. Build a corpus of photographed Sinhala newspaper pages with manual ground truth, and derive from it a frequency weighted taxonomy of the recognition errors that actually occur.
- O2. Generate a synthetic training corpus of noisy and clean sentence pairs from that measured error distribution, since no annotated parallel corpus exists.
- O3. Fine-tune a context-aware transformer corrector on the synthetic corpus and evaluate it on a test set of real photographed pairs, locked and split at photograph level before any model is trained.
- O4. Implement and evaluate the proposed detector-gated architecture against a full-sequence control under a win condition fixed in advance, and, if it fails, identify the cause rather than reporting the failure alone.
- O5. Reproduce classical dictionary and n-gram correction methods on the same test set, so that the necessity of a context-aware model is demonstrated rather than assumed.
- O6. Determine the capture conditions under which correction has anything to work with, and enforce them at capture time in a handset application usable without sight.
- O7. Integrate the component into the four-component assistive reading system behind a documented contract, such that no sibling component's failure can interrupt reading.
- O8. Validate with listeners that the measured improvement in error rate corresponds to an improvement in what a person actually understands.

All eight objectives are met and are reported in Chapters 3 to 6. Objective O8 was completed with six participants; Section 6.3 reports it with the limitations that bound a study of that size.

1.6 Scope and Delimitations

The scope of this component is the path from a photographed page to corrected Sinhala text: frame selection, region isolation, recognition, correction, a judgement of whether the result is fit to be spoken, and the handover to the components that answer questions

about it and speak it. The capture application is within scope because the capture requirement is a consequence of the research finding and cannot be delegated.

Six delimitations bound the work.

- One recognition engine. All errors measured in this work come from Tesseract [10] with the Sinhala trained data. Transfer to another engine's error distribution is untested.
- One document domain. All material is Sinhala newspaper text. Books, forms, signage and handwriting are out of scope.
- Correction operates on recognised text, not on the image. A corrector that could see the page would have evidence this one does not, and Section 9.3 identifies that as future work.
- Article segmentation of a full newspaper page is a sibling component's responsibility; this component assumes article level input and isolates a single article region from a close range photograph.
- The system is a research prototype. It holds one reader at a time in process memory, and is not a deployable multi user service.

2 LITERATURE REVIEW

2.1 Review Method and Scope

The review was conducted as a structured, non-exhaustive search rather than a formal systematic review, which is appropriate given the breadth of the three literatures involved and the size of the project. Three bodies of work were searched: assistive reading for blind and low-vision users, post-OCR error correction, and Sinhala and Indic language technology. Searches were run on the ACL Anthology, IEEE Xplore, the ACM Digital Library, arXiv and Google Scholar, with no lower date bound and a preference for the most recent work where a topic had been treated repeatedly.

Roughly 120 records were screened by title and abstract, of which 39 are cited. Two exclusions bound what this chapter can claim. Work published only in Sinhala was not searched, so local work that never reached an English-language venue may be missed. And where a preprint is cited it is identified as such in the reference list, because several of the most recent and most relevant results in this area have not yet been through review.

2.2 Assistive Reading for Blind and Low-Vision Users

2.2.1 The population and what it asks for

The World Health Organization reports that at least 2.2 billion people have a vision impairment, of whom at least one billion have an impairment that could have been prevented or has not been addressed [11]. The burden falls disproportionately on low- and middle-income settings, which is also where most low-resource languages are spoken. Together those facts describe the population this work serves: people who cannot read print, in a language commercial assistive technology does not cover.

What that population wants has been measured rather than assumed. Gamage et al. reviewed 646 papers on computer vision for blind and low-vision users against interviews with 24 participants [1]. They report only a weak positive correlation between what researchers work on and what users rate as important, and that 82 percent of the reviewed studies involved no blind or low-vision participants at all. Text recognition was the most-wanted capability, named by 15 of 20 participants. Reading printed text aloud is therefore the capability the population names first.

2.2.2 What existing reading aids achieve, and where they stop

Granquist et al. evaluated two leading commercial reading aids, a wearable device and a smartphone application, in a controlled study [4]. On flat, plain word-processed documents both exceeded 95 percent text-recognition accuracy. On formatted text and curved surfaces accuracy fell to between 13 and 57 percent, and task completion was 71 percent and 55 percent respectively, a difference that was not statistically significant.

Two conclusions follow. The hard case is not the clean document but the real one, and a newspaper page photographed by hand is firmly in the regime where these systems lose most of their accuracy. And this evidence concerns English: neither device offers Sinhala, so the measured limitations of commercial aids describe a ceiling a Sinhala reader cannot even reach.

2.2.3 Photographing a document without sight

A reading aid that depends on a photograph inherits the problem of taking one. Cutter and Manduchi compared three interaction modes for assisting capture and found that spoken guidance was 33 percent faster than fully automatic capture in one study, that 84 percent of failures came from holding the camera too close, and that short directional instructions outperformed detailed metric feedback [12]. The most striking result is a training effect: after using interactive guidance, participants' unaided capture accuracy rose from 71 to 86 percent in one study and from 64 to 94 percent in another.

Vazquez and Steinfeld reached a convergent conclusion from a different direction: comparing speech, tone and silent feedback for aiming a camera, they found speech guidance produced significantly better framing and received the highest usefulness ratings [13]. One practical detail bears on the design in Chapter 5: the instruction to move up is ambiguous between translating the device and tilting it, and ambiguous instructions are worse than none. What this literature does not supply is the quantity the guidance should be driven by. None of it specifies how large the characters must be for recognition to succeed, which is the gap Section 2.7 returns to.

2.2.4 The spoken channel, and what errors cost in it

Sinhala speech synthesis exists. Nanayakkara et al. reported a unit-selection system evaluated with 20 participants, scoring approximately 70 percent on both intelligibility and naturalness and approximately 60 percent on overall speech quality [14]. The

speech channel is therefore already imperfect, so text handed to it with one word in three wrong compounds two sources of difficulty for the listener.

The most pointed evidence comes from an adjacent population. Kafle and Huenerfauth found that word error rate correlates poorly with whether a reader actually understands error-containing output, reporting a Spearman correlation of 0.866 between an importance-weighted metric and users' subjective quality judgements against 0.108 for word error rate on the same data [15]. The population differs from the one considered here, but the structural situation is the same: text arriving serially, with no opportunity to re-read. An improvement in word error rate is therefore necessary rather than sufficient evidence that a listener is better served, which is why Section 6.3 exists.

2.3 Optical Character Recognition for Sinhala

2.3.1 The script and the difficulty it creates

Sinhala is an abugida occupying the Unicode block U+0D80 to U+0DFF. Each consonant carries an inherent vowel which is modified by dependent vowel signs attached above, below, before or after the base glyph, and suppressed by a dedicated sign. A large part of the information distinguishing one word from another is therefore carried in marks that are small relative to the base character, and that are the first thing lost when resolution, focus or contrast degrade.

de Silva maintains a survey of publicly available Sinhala natural language processing tools and research, including recognition applications [16]. Its framing is the one adopted here: Sinhala is resource-poor, having neither the economic pull of English nor the speaker scale of the largest languages.

2.3.2 Recognition engines and their reported accuracy

Tesseract is the engine used in this work, and its architecture is described by Smith [10], with the later extension that brought it beyond Latin script to the writing systems that include Sinhala [17]. It is the engine on which the Sinhala post-processing literature is built, and it is the baseline measured in Chapter 6.

Recent comparative work establishes where Tesseract sits among alternatives. Jayatilleke and de Silva evaluated six recognition engines in zero-shot mode on Sinhala and Tamil across five metrics, building on 6,969 Sinhala synthetic image-text

pairs [18]. For Sinhala the strongest engine measured a character error rate of 0.76 percent and a word error rate of 2.61 percent.

2.3.3 The gap between benchmark and page

The single most important observation in this section is that those favourable figures are measured on synthetic or clean material and do not survive contact with real degraded documents. Dilhara and Jayatilleke evaluated recognition on 1,010 annotated page images from Sri Lankan legislative acts spanning 1981 to 2019 [8]. On that authentic material the engine that measured 0.76 percent character error on synthetic data measured 8.84 percent, while Tesseract measured 10.69 percent and the best fine-tuned system reached 1.05 percent.

A tenfold gap between a synthetic benchmark and an authentic page is the empirical foundation of this dissertation. A claim about Sinhala recognition accuracy is almost meaningless without a statement of what was photographed and how, and post-OCR correction is not an optional refinement on top of an already-solved problem: on real pages the error rate remains high enough that a listener cannot follow the text.

2.4 Post-OCR Error Correction

2.4.1 The problem and its standard decomposition

Post-OCR correction takes the output of a recognition engine and repairs it using knowledge about the language rather than about the image. Nguyen et al. survey the field and organise it into an error detection stage and an error correction stage, reviewing lexical, statistical, machine-learning and neural approaches together with the available datasets and metrics [19]. That two-stage decomposition is not merely descriptive: it is the conceptual ancestor of the detector-gated architecture this dissertation evaluates.

The field has shared benchmarks. The ICDAR 2017 competition split the problem explicitly into detection of erroneous character sequences and correction of those sequences, and established the aligned recognised and gold-standard corpus format that later work reuses. The 2019 edition expanded to roughly 22 million recognised characters with aligned gold standard across ten European languages [20]. No comparable resource exists for Sinhala.

2.4.2 Lexical and statistical approaches

The classical formulation treats recognition as a noisy channel. Kolak et al. presented a generative probabilistic model describing the full process from true text to corrupted recognition output, and evaluated it on French text deliberately processed with an English recognition system [21]. They report a 70 percent reduction in word error rate and a 41.92 percent reduction in character error rate. The important idea for this work is that the corruption model is learned rather than hand-written, which is the direct ancestor of synthetic error generation.

For Sinhala specifically, two pieces of prior work define the classical baseline. Wasala et al. built a data-driven spell checker using n-gram statistics, deliberately avoiding deep linguistic resources on the grounds that Sinhala's morphological complexity makes exhaustive lexical enumeration impractical, and report an average accuracy of 82 percent on common spelling errors [6]. Balasooriya built a post-processing pipeline over Tesseract for Sinhala comprising a syntactic rule engine, character-pair confusion detection and dictionary-based word-level correction, raising word-level accuracy from 53.22 percent to 73.27 percent after Unicode normalisation and to 86.16 percent after confusion-pair correction [7].

Balasooriya's work deserves emphasis for two reasons. It establishes that Sinhala post-OCR correction is not new, so the present work does not claim to be first. And its appendix lists the visually similar character pairs that Tesseract conflates, including the consonant pair that Chapter 6 measures as the single most frequent substitution in this corpus. An error list compiled by a different author, on different material, several years earlier, agreeing with the one measured here is corroboration of a kind rarely available.

What the classical approaches share is a dependence on the word: both decide whether a word is wrong by looking at the word. Section 2.5 gives the quantitative reason that fails for morphologically rich Indic languages.

2.4.3 Neural sequence-to-sequence correction

The neural turn recasts correction as translation from erroneous text to correct text. Soper et al. fine-tuned a pretrained denoising sequence-to-sequence model on historical newspaper recognition output and reported a 29.4 percent improvement in

character accuracy, presenting it as the first application of sentence-level transformer models to the task [22].

Lyu et al. built a character-level model combining recurrent and convolutional components with a custom loss designed to encourage the model to actually change text, reporting a reduction of more than 89 percent in word error rate from a 32.3 percent baseline on German historical prints [23]. The custom loss addresses the same tension this dissertation studies from the opposite side: their model had to be pushed towards changing text, while a detector gate pushes a model towards changing less.

Two pretrained models recur throughout this literature. mT5 is a multilingual text-to-text transformer pretrained on a Common Crawl corpus covering 101 languages including Sinhala, and it is the base model fine-tuned in this work [24]. ByT5 operates directly on raw bytes, matching token-level performance while being markedly more robust to noise and requiring no language-specific tokenisation [25]. The byte-level property is attractive for a script whose subword tokenisation is poorly supported, and Section 9.3 identifies the comparison as the most valuable single extension to this work.

2.4.4 Synthetic training data

The central obstacle for a low-resource language is that no annotated parallel corpus of recognised and correct text exists. The field's answer is to synthesise one, and the most directly comparable study is Guan and Greene, who compared synthetic data strategies across volume, augmentation and generation method [26]. Their contribution is an algorithm that uses computer-vision feature detection to compute glyph similarity and synthesises realistic errors from it. Models trained purely on this synthetic data significantly reduce character error rate, with results reported for seven languages at different resource levels. Those figures are the comparison used in Section 7.3.

2.4.5 Detection-then-correction architectures

This is the sub-area the present work contributes to most directly, and it is strikingly thin. The decomposition into detection and correction is standard in surveys and competitions, but very few papers isolate the architecture and measure whether gating correction on a detector actually helps.

The principal positive result is Schaefer and Neudecker, who present an explicitly two-part system in which a first component detects erroneous sequences and a second corrects only those [9]. They report that applying the detection model ahead of correction reduces both the character error rate, relative to a one-step correction model, and the number of correct characters that are falsely changed. That claim is the one the negative result of this dissertation stands against, and Section 7.2 locates the discrepancy in the differences between the two settings rather than asserting that either measurement is wrong.

Nguyen et al. combined a contextual language model with neural machine translation to perform both detection and correction, reporting results comparable to the leading methods on the English data of the two ICDAR competitions [27]. It is the closest architectural precedent for a BERT-style detector feeding a sequence-to-sequence corrector, which is the architecture evaluated here.

The thinness of this sub-area is itself a finding. A negative result on detector-gated correction contradicts essentially one prior positive result, obtained on different languages with a different detector, rather than overturning a settled consensus.

2.4.6 Large language models as correctors

The newest line of work prompts general-purpose language models to correct recognition output. Thomas et al. compared instruction-tuned generative models against a fine-tuned denoising sequence-to-sequence model on a corpus of nineteenth-century newspaper text and reported a 54.51 percent reduction in character error rate for the generative model [28].

The qualification is as important as the result. Kanerva et al. evaluated open-weight language models for post-correction on historical English and Finnish and report that while such models reduce character error usefully in English, practically useful performance for Finnish was not reached [29]. Methods that work for English therefore do not automatically transfer to a morphologically rich, lower-resource language, which is the situation Sinhala is in; and reporting such a failure is a contribution rather than an embarrassment, which is the position taken in Section 7.4.

The design decision in Chapter 4 to disable general-purpose language-model post-editing by default is informed by this literature rather than opposed to it. A model that

rewrites recognised text may improve it, but once it has done so the quantity being measured is no longer the correction model the research is about.

2.5 Post-OCR Correction for Low-Resource and Indic Scripts

The closest body of work to this dissertation concerns the other scripts of South Asia, which share Sinhala's abugida structure and its resource scarcity.

Sankaran and Jawahar provide the quantitative explanation of why word-level methods fail for these languages [5]. They report word-uniqueness rates of 29.83 percent for Malayalam and 25.62 percent for Telugu against 4.93 percent for English; that a dictionary covering 80 percent of language use needs roughly 300,000 words for those languages against 8,000 for English; and that over a third of Indian language words become another valid word under a single character substitution, against 5 percent for English. A dictionary cannot detect an error that produces another real word, which is the strongest available argument for context-aware correction over lexical lookup in Sinhala.

Krishna et al. corrected the output of recognition engines built for other Roman-script languages applied to Romanised Sanskrit, and showed that a copying mechanism in the sequence-to-sequence model lifted accuracy from 35.76 to 87.01 percent in one setting [30]. The copy mechanism matters because most tokens arrive correct and the model's dominant behaviour must be to leave them alone, which is the same requirement that motivates detector gating.

Maheshwari et al. released a Sanskrit post-OCR correction benchmark of approximately 218,000 sentences drawn from 30 books, and identified three dominant error sources: word-boundary detection, diacritics, and orthographically similar characters [31]. Their best system, a byte-level model with phonetic encoding, reduced character error from 3.89 to 2.98 percent and word error from 30.23 to 23.19 percent. The error taxonomy is almost exactly the one measured for Sinhala in Chapter 6, which is evidence that the structure of the problem is a property of the script family rather than of this corpus.

Kashid and Bhattacharyya generate synthetic post-OCR training data for Devanagari languages by rendering clean text in 50 fonts and passing it through a recognition engine, treating the resulting errors as mistranslations in a parallel corpus [32]. They release approximately 16.2 million sentence pairs across six languages and report, for

Sanskrit, a reduction in character error from 8.77 to 5.67 percent. This is the closest methodological analogue to the present work: the same problem of absent gold data in a low-resource Indic script, and the same solution shape. Sinhala is explicitly not among the languages covered.

A recent stress-test benchmark for Devanagari reports that under controlled blur, noise and low-resolution conditions, classical engines fail on numerals and punctuation while vision-language models fail on conjuncts, vowel diacritics and nukta marks, and that strong English performance does not predict Indic performance [33]. That diacritics are the first casualty of degradation is the cross-script corroboration of the optical ceiling argument developed in Section 7.1.

2.6 Sinhala Language Resources

Building a correction model requires pretrained representations of the language as well as a corpus of errors. Dhananjaya et al. provide the most comprehensive comparison of pretrained language models for Sinhala, benchmarking multilingual models against two new monolingual Sinhala models pretrained on 15.7 million sentences and approximately 192.6 million words assembled from Common Crawl, OSCAR and raw Sinhala news text [34]. The larger of the two monolingual models is the token classifier used as the detector in this work. The my own finding that the larger variant gains little over the smaller is relevant to any replication.

Taken with the survey of Sinhala tools [16], the picture is of a language with real but thin resources: pretrained encoders, a spell checker, speech synthesis and recognition engines all exist. What does not exist is an annotated post-OCR correction corpus, a measured error taxonomy from real photographs, or a shared benchmark on which competing correction methods could be compared. Those absences define what this dissertation contributes.

2.7 Image Quality, Capture and Multi-Frame Methods

2.7.1 How much resolution recognition needs

The capture requirement this work adopts is expressed as a minimum glyph height in pixels, and an honest statement about the literature is required here: no peer-reviewed study was found that specifies such a minimum. The widely circulated figures for minimum character height and scanning resolution originate in vendor documentation

rather than academic work, and are not cited here as research findings. The closest defensible evidence is indirect: the Devanagari stress-test benchmark shows that under degradation it is precisely the diacritic marks that fail first in an abugida [33]. It does not supply the number this work needs, which is why Chapter 3 measures it.

2.8 Evaluation Practice and its Pitfalls

Two methodological issues recur in this literature and both shaped the design of the present study.

The first is comparability. Absolute character error rates depend on the recognition engine, the document condition and the test material, so the field compares relative error reduction instead [19]. Even that comparison must be read with care, because a reduction measured against a weak engine on clean scans is not the same achievement as the same reduction against a stronger engine on photographed newsprint. Section 7.3 states both the comparison and its limits.

The second is the construction of the evaluation set. Post-OCR evaluation requires aligning recognised text with reference text, and a tempting hygiene step is to discard pairs whose alignment quality is poor. Section 7.4 reports that such filtering is selection on the dependent variable, because a pair aligns poorly precisely when recognition failed badly. This dissertation reports word error rate because it is the field's comparable metric, and supplements it with a listening study because the field's metric is not the quantity the application is trying to improve.

3 RESEARCH METHODOLOGY

3.1 Research Design

The study is a quantitative, experimental comparison of five text-correction systems evaluated on a single held-out test set of real photographed Sinhala newspaper text. The independent variable is the correction method applied to recognised text; the dependent variables are character error rate, word error rate, the proportion of individual error operations repaired, and the number of already-correct words destroyed. All five systems receive identical input and are scored by identical code. Three properties of the design are stated before the detail, because the credibility of every later result depends on them.

First, the test set was locked before any model was trained. The page list, the random seed and a LOCKED flag were written to a manifest at the end of corpus construction, and the test partition was never used for training, for validation, or for selecting the detector operating threshold. This removes the most common source of optimistic bias in small-corpus machine learning studies: the gradual leakage of test information into modelling decisions through repeated inspection.

Second, the split is at photograph level rather than at sentence level. Sentences extracted from the same photographed article share the same typeface rendering, print quality, paper condition, illumination and camera geometry. A sentence-level split would evaluate the model on capture conditions it had already seen, and the resulting figure would describe memorised conditions rather than generalisation. Splitting at photograph level removes that possibility at the cost of a smaller effective dataset, and the cost is accepted deliberately.

Third, the study was designed so that the proposed architecture could fail. The component under investigation is a two-stage, detector-gated corrector in which a token classifier flags words containing recognition errors and a sequence-to-sequence model corrects only the flagged spans; a single-stage, full-sequence corrector was implemented as the control. The condition under which the proposed architecture would have been accepted was fixed in advance: a lower character error rate than the control on the locked test set. That condition was not met. The pre-registered win condition is what allows the negative outcome to be read as a finding rather than as a failure of execution.

The design addresses the research questions of Section 1.5 as follows. Whether a transformer-based corrector trained on synthetically noised text can reduce error rates on real photographed Sinhala newsprint is answered by the comparison between raw recognition and the full-sequence corrector in Section 6.1. Whether detector-gated correction outperforms full-sequence correction is answered by the direct comparison of the two neural systems and by the threshold sweep of Section 3.5. Which categories of recognition error are actually repaired is answered by the per-category analysis defined in Section 3.4.3.

The Topic Assessment Form committed to generating noisy and clean sentence pairs, training a transformer model on them, and evaluating with character and word error rate using a small number of real recognition examples. This study follows that plan but strengthens its evaluation: instead of a small number of real examples, the evaluation uses 217 real photographed sentence pairs on a locked, page-disjoint partition. The strengthening is deliberate and is recorded here so the divergence from the registered plan is visible.

3.2 Data and Study Materials

3.2.1 Source material and capture

The corpus consists of 232 close-up photographs of individual articles taken from three Sinhala daily newspapers, Dinamina, Lankadeepa and Divaina. Each photograph frames a single article rather than a full page, which reflects the way the deployed system is used: a reader points a hand-held camera at the article they wish to hear. Photographs were taken hand-held under ordinary indoor and daylight conditions, with no studio lighting, no copy stand and no flattening of the newspaper, so the images carry the perspective distortion, uneven illumination, focus variation and paper curl that a deployed system must tolerate.

Ground truth was produced by manual transcription. 230 photographs were transcribed by the me character by character against the printed original, including punctuation and digits; two were excluded because the printed text could not be transcribed with confidence. Hyphenated words broken across a line were joined, so the ground truth represents the words of the article rather than the typesetting of the column. Transcription is the most time-consuming element of the study and is the reason the corpus is of this size.

3.2.2 Optical character recognition

Each photograph was recognised with Tesseract 5.5.3 [10] using the Sinhala trained data, invoked through the pytesseract interface at its default engine mode and default page segmentation mode. Tesseract was selected rather than a more accurate contemporary engine for two reasons that are properties of the deployment: it runs offline on modest hardware, which the wearable reading device this component supports requires, and it does not depend on a served model or a container runtime. Section 2.3 discusses the accuracy given up by this choice and Section 7.5 records it as a limitation.

The recognition settings were held constant across the whole corpus. No preprocessing, binarisation, deskewing or contrast adjustment was applied before recognition at this stage, so the measured error rates describe the recognition problem as it occurs rather than the effect of a preprocessing pipeline. Capture-side optimisation is treated separately in Section 6.2.

3.2.3 Alignment and sentence-pair extraction

The unit of evaluation is a sentence pair: a span of recognised text together with the transcribed sentence it corresponds to. Producing these pairs requires aligning two strings that differ, and the procedure is described in detail because an earlier version of it introduced a serious bias, reported in Section 7.4 as a methodological finding.

Both texts are first normalised to Unicode normalisation form C. Zero-width joiners, zero-width non-joiners, soft hyphens and byte-order marks are removed, line-break hyphenation is resolved, and runs of whitespace are collapsed to a single space. The normalised transcription is then segmented at sentence terminators. For each transcribed sentence the corresponding recognised span is located by character-level alignment: every position at which the two strings agree is recorded as an anchor, and the span bounded by the first and last anchor inside that sentence is extracted and extended to the nearest word boundaries. A candidate pair is retained only with at least three anchors and an anchor density of at least 0.10.

Each surviving pair is then classified by three measures: the match fraction, the proportion of the transcribed sentence covered by matching blocks; the length ratio between the recognised span and the transcribed sentence; and the fraction of Latin characters in the recognised span. A match fraction below 0.45 is classified

MISALIGNED, a Latin fraction above 0.15 is classified SCRIPT, a length ratio below 0.85 is classified TRUNCATED and one above 1.20 BLOAT. Everything else is classified CORRECTABLE. Table 3.1 gives the resulting distribution over the whole corpus.

Table 3.1: Classification of aligned sentence pairs across the 230 transcribed photographs

Classification	Pairs	Percentage	Meaning
CORRECTABLE	489	88.4	Recognised span corresponds to the sentence
BLOAT	36	6.5	Span substantially longer than the sentence
TRUNCATED	20	3.6	Span substantially shorter than the sentence
MISALIGNED	8	1.4	Span does not correspond to the sentence
SCRIPT	0	0.0	Output predominantly non-Sinhala
Total	553	100.0	

Of the 489 pairs classified CORRECTABLE, the 482 whose transcribed sentence contains between three and sixty words form the working set. The lower bound removes fragments too short to carry context; the upper bound removes sentences that would be truncated by the model's maximum input length. These 482 pairs come from 218 of the 230 transcribed photographs, the remaining twelve having produced no pair that satisfied the criteria.

The classification is applied once, at corpus construction, and the complete distribution is reported in Table 3.1 rather than being summarised as a retention rate. Only the 1.4 percent classified MISALIGNED are removed on a similarity criterion, because a misaligned pair measures the alignment procedure rather than the recognition quality. Nothing is filtered after the split is locked, and no pair is removed from the test partition on any grounds.

3.2.4 The locked evaluation split

The 218 photographs that produced usable pairs were shuffled with random seed 42 and divided at photograph level into training, development and test partitions in the proportions 40, 15 and 45 percent. The larger test proportion is a deliberate choice for a small corpus: the study's purpose is measurement rather than the production of the strongest possible model, and a larger test partition narrows the confidence interval on the quantity being measured. The partition manifest, containing the three page lists, the seed and a LOCKED flag, was written before any model was trained.

Table 3.2: The locked, page-disjoint evaluation split

Partition	Photographs	Sentence pairs	Raw-OCR CER
Training	87	187	0.1141
Development	33	78	0.1210
Test (locked)	98	217	0.1197
Total	218	482	0.1177

The final column of Table 3.2 is the baseline character error rate of raw recognition within each partition, included as evidence that the split is not itself biased. The three values lie within 0.007 of one another, so no partition is systematically easier or harder than the others.

The test partition contains 217 sentence pairs drawn from 98 photographs, comprising 28,217 ground-truth characters and 4,348 ground-truth words. It consists entirely of real photographed text; no synthetic data appears in it at any point.

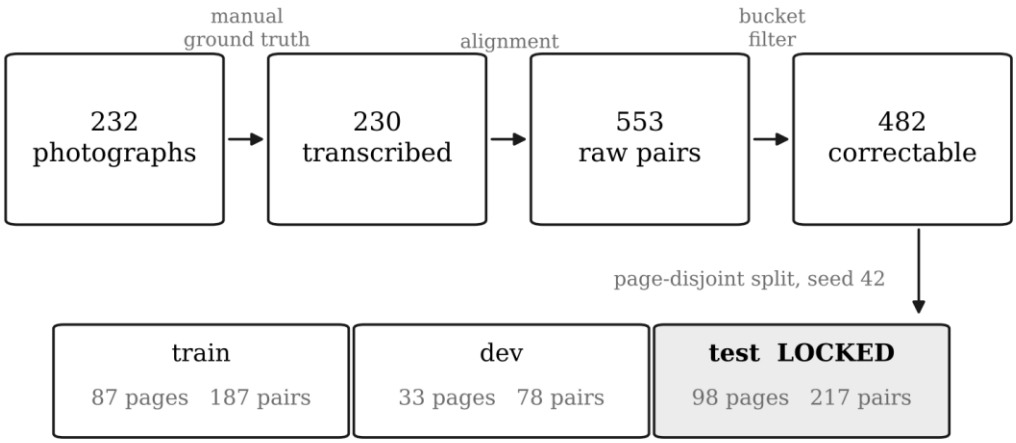


Figure 3.1: Corpus construction and the locked page-disjoint split

3.2.5 Synthetic training corpus

No annotated post-OCR correction corpus exists for Sinhala, and the 187 real training pairs available from the locked split are far too few to fine-tune a sequence-to-sequence model. The training data is therefore generated synthetically, following the approach established for other low-resource languages [32]: clean text is corrupted according to a model of the errors a recogniser makes, and the model learns to invert that corruption.

The clean text is a corpus of 24,212 Sinhala sentences scraped from the public websites of three news organisations. Table 3.3 gives the composition. The imbalance toward a single source is a limitation recorded in Section 7.5: its practical effect is that the model

is trained predominantly on text from one publication and evaluated on photographs of two others, which makes the task harder rather than easier.

Table 3.3: Composition of the scraped clean-text corpus

Source	Sentences	Percentage
Adaderana	22,506	92.96
Divaina	878	3.63
Lankadeepa	828	3.42
Total	24,212	100.00

The scraped text is normalised to form C, stripped of invisible characters and deduplicated, leaving 24,209 distinct sentences. A leakage filter is then applied: any scraped sentence whose similarity to a held-out transcription from the development or test partition reaches 0.70 is removed. The filter is necessary because news organisations publish online versions of printed articles, so without it a test sentence could enter training in near-identical form and the reported error rate would be meaningless. The filter is applied in one direction only, from the scraped corpus toward the held-out data, so no held-out material is modified.

3.3 Study Procedure

3.3.1 Construction of the error taxonomy

The corruption applied to the clean text is not invented. It is derived from a taxonomy of the recognition errors actually observed in this corpus, which is itself one of the contributions of this work.

The taxonomy is measured from the 265 pairs in the training and development partitions only; the test partition is excluded, because a noise model estimated from the test data would make the training distribution a function of the evaluation data. Every non-matching alignment operation is recorded: a substitution replaces a span of at most three characters with another such span, a deletion removes a span of at most two characters, and an insertion adds one. Longer operations count toward the overall error rate but are not entered into the confusion tables, because a long replacement reflects alignment uncertainty rather than a single character confusion.

The procedure yields 192 distinct substitution types, 29 distinct deletion types and 92 distinct insertion types, each with an occurrence count, at an overall character error rate of 0.1092 across the source pairs. Table 3.4 gives the most frequent members of

each group. The dotted circle in the table marks a dependent sign that attaches to a consonant base and is shown here in isolation.

Table 3.4: Most frequent measured confusions, with occurrence counts

Type	Ground truth	Recognised as	Count
Substitution	◌ඛ	◌ඛ	118
Substitution	◌ු	◌ෘ	91
Substitution	◌ැ	◌ෙ	65
Deletion	◌ොා	(removed)	30
Deletion	◌ෙ	(removed)	25
Deletion	◌ැ	(removed)	18
Insertion	(none)	underscore variant	55
Insertion	(none)	underscore variant	53

Two properties of this distribution should be stated together, because either alone gives a misleading impression of where the difficulty lies. The single most frequent confusion is a consonant pair, ඛ recognised as ඛ, which occurs 118 times. In aggregate, however, dependent vowel signs form the largest error category, as the per-category analysis in Section 6.1 shows. The most frequent insertions are not character confusions at all: they are underscore variants produced when the recogniser interprets a rule line or a column margin as text.

3.3.2 Generation of synthetic training pairs

Each clean sentence selected for corruption is processed word by word. A target proportion of words is chosen for corruption, calibrated so that the generated word error rate matches the real word error rate measured on the training and development pairs, which is 0.273. Within each selected word, between one and three operations are applied, with probabilities 0.50, 0.35 and 0.15 respectively, so that most corrupted words carry a single error and a minority carry the compound damage that occurs in practice.

Each operation is drawn as follows. With probability 0.40 the operation is sampled from the measured substitution table of Section 3.3.1, weighted by occurrence count, so that frequent real confusions appear frequently in training. Otherwise the operation substitutes a dependent vowel sign for another sign within the same Unicode-defined modifier group, deletes a character, or inserts a modifier. Restricting random substitutions to within-group alternatives keeps the generated noise plausible: an

arbitrary character substitution produces damage no recogniser would make, and a model trained on it learns to repair a problem that does not occur.

Two further provisions were made. A further 40 percent of the corpus is emitted unchanged, as identity pairs in which input and target are the same sentence; without these, a model trained only on damaged input learns that every input requires modification and alters text that was already correct. The 187 real training pairs are upsampled twentyfold so that genuine recognition errors are not overwhelmed by synthetic ones. The procedure produces 36,743 sentence-level training pairs and 179,931 span-level training examples for the gated corrector.

3.3.3 Systems compared

Five systems are evaluated on the locked test set. All five receive the same raw recognised text and are scored by the same code.

B1, raw recognition, is the recognised text with no correction applied. It establishes the baseline against which every other system is measured and represents what the user would hear if the correction component did not exist.

The dictionary baseline performs lexicon lookup with edit-distance correction. A vocabulary of 42,731 distinct word types is built from 448,387 word tokens of the scraped corpus of Section 3.2.5. A word already present in the vocabulary is left unchanged; a word absent from it is replaced by the nearest vocabulary entry within edit distance two, searching only entries whose length differs by at most two characters, with ties broken by corpus frequency. If no candidate lies within edit distance two the word is left unchanged. This represents the lexicon-based approach that dominates the prior work on Sinhala recognition correction.

The n-gram baseline performs statistical spelling correction. A character trigram model is estimated from the same vocabulary with sentence-boundary padding and add-one smoothing, with each word's trigrams weighted by its corpus frequency. For an unknown word, candidates within edit distance two are scored by the mean log trigram probability of the candidate plus a small term proportional to the log of its frequency, and a bias is applied against changing the input. This represents the statistical spell-checking approach reported for Sinhala [6].

B3, the full-sequence corrector, is mT5-small [24] fine-tuned on the synthetic sentence pairs of Section 3.3.2. The complete sentence is passed to the model and a corrected

sentence is generated. No task prefix is used and no inline markers are placed around suspected errors; the absence of markers follows an earlier experiment in which inline error markers caused the model to rewrite entire sentences and to generate content not present in the input.

The proposed system is the detector-gated corrector. A SinBERT-large [34] token classifier assigns each word a probability of containing a recognition error. Each word whose probability reaches the operating threshold is presented to a span-level mT5-small corrector as its four preceding words, the word itself delimited by bracket markers, and its four following words. The generated string replaces the word if it is at most three tokens long, a guard that prevents a single flagged word from being replaced by a phrase. Words not flagged are copied into the output verbatim and are never seen by the corrector.

The rationale for the gated design is a safety property specific to this application. The output of this component is spoken to a user who cannot see the page, so if the corrector rewrites an already-correct word into a different word that sounds plausible, the user has no means of detecting the substitution. A design in which unflagged text cannot be modified bounds that risk by construction.

3.3.4 Selection of the detector operating threshold

The gated system requires a probability threshold above which a word is flagged. The threshold was selected on the development partition only, by a rule stated before the curve was examined: choose the highest recall subject to a false-positive rate of at most 0.10 on real text, where the false-positive rate is the proportion of words that are in fact correct but are nevertheless flagged.

The rule favours precision over recall deliberately, and the asymmetry follows from the application. A word the detector misses is left in the state recognition produced it, which is the state it would have been in had the component not existed. A correct word the detector wrongly flags is exposed to a corrector that may change it. Under-flagging therefore fails safe and over-flagging does not. Selecting by F1 score, which weights the two errors equally, was used in an earlier iteration and contributed to its failure.

Applying the rule to the 78 development pairs selects a threshold of 0.30, at which the detector achieves a recall of 0.6512, a precision of 0.9461 and a false-positive rate of 0.0157.

3.3.5 Training configuration

Table 3.5 gives the training configuration of the three models. All training was performed on a single NVIDIA T4 graphics processor. Mixed-precision training is disabled for both sequence-to-sequence models because mT5 produces non-finite losses under half precision on this hardware; it is enabled for the token classifier, which is numerically stable under it. Zero-width joiners, zero-width non-joiners and soft hyphens are stripped before tokenisation, as their presence caused training failures. The token classifier labels only the first sub-token of each word, with the remaining sub-tokens masked out of the loss, so that long words do not contribute more to the objective than short ones.

Table 3.5: Training configuration of the three models

Parameter	Full-sequence corrector	Span corrector	Detector
Base model	mT5-small	mT5-small	SinBERT-large
Task	seq2seq	seq2seq	token classification
Training examples	36,743	179,931	36,743
Maximum length	128 tokens	128 tokens	128 tokens
Learning rate	5×10^{-4}	5×10^{-4}	2×10^{-5}
Warmup steps	300	300	200
Weight decay	0.01	0.01	0.01
Batch size	16	16	16
Epochs	3	2	3
Mixed precision	disabled	disabled	enabled
Random seed	42	42	42

3.3.6 Inference configuration

All results reported in this thesis were generated with beam search using four beams and a maximum output length of 128 tokens, held identical across both neural systems so that the comparison between them is not confounded by generation settings. Section 6.2 reports a separate experiment on the effect of a repetition constraint during decoding, conducted under a different normalisation convention; its figures are not interchangeable with those of Section 6.1.

3.4 Measures and Evaluation Metrics

3.4.1 Character and word error rate

Character error rate and word error rate are the primary measures. Both are computed from the same alignment procedure used elsewhere in this work, so that a reader comparing a reported rate against an aligned example sees consistent counts.

Let D(h, r) denote the set of non-matching alignment operations between a hypothesis string h and its reference string r. The cost of an operation is the larger of the number of hypothesis symbols and the number of reference symbols it spans, which charges a substitution once rather than twice and charges an insertion or deletion by its length. The total cost for a sentence, c(h, r), is the sum of these operation costs; the rate for that sentence is the total cost divided by the length of the reference, and the reported figure is the arithmetic mean over the N = 217 test sentences:

$$CER = (1/N) \Sigma_i \, c(h_i, r_i) / |r_i| \qquad (3.1)$$

Word error rate is computed identically with the symbol unit set to whitespace-delimited words rather than characters. The averaging is at sentence level, not at corpus level, so that a long sentence does not dominate the reported figure. This choice is recorded because it affects the value: a corpus-level micro-average over the same data gives a different number, and the two must not be compared.

3.4.2 Over-correction

Error rate alone does not distinguish a system that repairs many errors from one that repairs few but damages nothing. Because the output of this component is spoken to a user who cannot verify it, damage to text that was already correct is a distinct harm and is measured separately.

For each test sentence, the set C is the set of recognised word types that align to the reference under a matching operation, that is, the words recognition already produced correctly. A member of C absent from the system output is counted as destroyed. Across the test partition, C totals 3,350 words, and the over-correction rate of a system is the number of destroyed words divided by that total. The measure is conservative in one respect: because it works on word types rather than tokens, a word that appears twice in a sentence and is damaged once is not counted.

3.4.3 Per-category error analysis

An aggregate error rate states how much a system improves text but not what it improves. Every error operation between raw recognition and the reference in the test partition was therefore extracted and assigned to exactly one category. The partition yields 1,329 error operations.

The categories are derived from the code-point classes of the Sinhala Unicode block, U+0D80 to U+0DFF, together with categories for punctuation, whitespace, Latin characters and digits, and non-script noise. Deriving them from the script's own encoding rather than by hand means every operation falls into exactly one category without a judgement being made, and the definitions are reproducible by anyone with access to the Unicode standard. The ten categories are vowel sign, consonant, mixed, noise artifact, punctuation, hal kirima, independent vowel, Latin or digit, space and anusvara.

An error operation counts as fixed by a system only when every reference character position it damaged is correct in that system's output; a partial repair does not count. Separately, operations a system creates on spans that recognition had produced correctly are counted as new errors, so that a system cannot improve its apparent fix rate by damaging text elsewhere.

3.4.4 Reachability of the gated architecture

The gated architecture can only repair errors in words its detector flags, so its attainable fix rate is bounded above by the detector's recall, and comparing achieved fix rate against that bound separates two very different explanations for a poor result. If the achieved rate is close to the ceiling, the limitation lies with the detector; if it is far below, the limitation lies with the corrector and improving the detector cannot help. The reachability ratio is defined as

$$\eta = f_G / \rho \qquad (3.2)$$

where f_G is the proportion of all error operations the gated system repairs and ρ is the detector's recall on the test partition. The quantity is reported in Section 6.1 and is the basis of the argument in Section 7.1.

3.4.5 Two measurement conventions

Two measurement conventions appear in this thesis and they are not interchangeable. The primary results of Section 6.1 use unnormalised text and a sentence-level arithmetic mean, the convention under which the locked evaluation was performed. The capture-side experiments of Section 6.2 use text normalised to form C with whitespace collapsed and an item-level micro-average, which produces systematically lower values on the same data. Figures from the two conventions are never placed in the same table.

3.5 Data Analysis

The difference in mean character error rate between two systems is assessed by a paired bootstrap over sentences with 10,000 resamples. On each resample, 217 sentence indices are drawn with replacement and the difference in mean error rate recorded; the reported interval runs between the 2.5th and 97.5th percentiles of the resulting distribution. A bootstrap is used in preference to a paired t-test because per-sentence differences in this data are strongly non-normal: most sentences show a small improvement, a minority a large one, and a few are made worse. The test is paired because both systems are evaluated on the same sentences.

For interpretation, the ten categories of Section 3.4.3 are also reported in three groups. Sinhala script errors comprise the vowel sign, consonant, hal kirima, independent vowel and anusvara categories. Formatting and non-script errors comprise the noise artifact, punctuation, Latin or digit and space categories, which arise from the recogniser interpreting page furniture as text. Multi-character operations spanning both groups form the third. The grouping is defined before any result is seen, so that it cannot be chosen to favour a conclusion.

Two further analyses bound the conclusions. To determine whether the gated architecture's result is attributable to its detector, the detection threshold is swept across nine settings from 0.1 to 0.9, which move the detector's recall on the test partition from 0.693 to 0.575, and both the character error rate and the over-correction rate are recorded at each. And to determine whether the corpus is large enough for the conclusions drawn from it, the full-sequence corrector was additionally trained on a reduced corpus and evaluated on the same locked test partition.

3.6 Ethics, Privacy, Security and Responsible Use of Automated Tools

The corpus consists of photographs of newspapers purchased at retail. The photographs and the transcriptions derived from them are used solely as research material and are not redistributed. No article text is republished in this thesis beyond the short extracts required to demonstrate a correction, which is consistent with fair dealing for research and private study. The scraped clean-text corpus of Section 3.2.5 was collected from publicly accessible web pages and is likewise not redistributed. In deployment the system reads a newspaper the user has in their possession, and does not reproduce, store or transmit the publication beyond reading it aloud to that user.

A photograph taken by a user of an assistive reading device may incidentally capture their surroundings, their hands, or material adjacent to the article they intended to read. The deployed system therefore retains no captured image: frames are held in memory for the duration of a single request and discarded when the response is returned. The session record that allows a follow-up question holds only the recognised and corrected text, expires after thirty minutes, and is bounded to thirty-two concurrent sessions. No user identifier, location or device identifier is stored.

On security, the component communicates with the other three components over local HTTP rather than by importing their code, and the service layer never raises an exception into the reading path, so a service that is unavailable, slow or returning malformed data degrades one feature rather than failing the system. Replies are validated against a required field set before being used, and any reply originating from a stub implementation is marked as such so that it cannot be reported as a model result.

Three commitments govern the reporting in this thesis. Every figure traces to a stored result file, and no figure is quoted from a summary document when the artefact that produced it is available. Where a measurement convention affects a value, the convention is stated at the point of use. Where a result is unfavourable to the proposed design, it is reported with the same prominence as a favourable one. Generative artificial intelligence tools were used during this project; their use and the verification applied to their output are disclosed in Appendix B. The research design, the experimental decisions and the conclusions are the my own.

The core experimental study involves no human participants: it operates on photographs taken by the me and on text transcribed by the me, and no personal data

is collected or processed. Section 6.3 describes the user-facing evaluation conducted for this work, together with the constraints that shaped it.

3.7 Reproducibility and Research Artefacts

A result that cannot be reproduced is an assertion rather than a finding. The evaluation split is stored as a manifest containing the page lists, the random seed and a LOCKED flag, written before modelling began. Per-sentence outputs for every system on every test sentence are stored, so any reported aggregate can be recomputed and any individual example inspected. The measured confusion tables, the per-category analysis and the threshold sweep are each stored as separate result files. Appendix A lists the released artefacts and what each contains.

4 PRODUCT AND SOLUTION DESIGN

This chapter states what the component is required to do, how it is structured to do it, and which alternatives were considered and rejected along the way.

4.1 Requirements Specification

4.1.1 How these requirements were derived

The requirements in this section were written after the component was built, not before it. This is stated plainly because presenting a retrospective specification as though it had guided the work would misrepresent the process. The project began from a research question rather than from a product specification, and the behaviours now recorded as requirements were discovered in sequence: some from the research result, some from the integration contract negotiated with the other three components, and several from defects whose repair is now pinned by a test.

The discipline applied was to admit a requirement only when it has a verification route that exists today. Every functional requirement below names either a test module and function in the suite described in Section 5.4, or a measurement reported in Chapter 6. Where a requirement is only partially satisfied, the table says so rather than rounding up.

The scope of these requirements is this component: capture, frame selection, region isolation, recognition, correction, readability assessment, the local interaction layer, and the payload handed onward. Requirements belonging to the sibling components appear only where this component must behave correctly when they are absent.

4.1.2 Functional requirements

Table 4.1: Functional requirements, acceptance criteria and verification

ID	Requirement and acceptance criterion	Verified by
FR1	Accept a capture burst over HTTP and return recognised, corrected Sinhala text as structured data. Every field the handset reads by name is present in every reply.	test_api: capture_returns_every_field_the_phone_reads; it_is_json_not_audio
FR2	Correct recognition errors in Sinhala text. Character error rate on the locked test set must be lower than uncorrected recognition by a margin whose 95% confidence interval excludes zero.	tools/verify_model.py; Chapter 6, Table 6.1

ID	Requirement and acceptance criterion	Verified by
FR3	Reduce a multi-frame burst to the frames worth recognising, and reconcile disagreement between frames line by line rather than discarding the minority.	test_vote_lines (7 cases, including a corrupted line outvoted by two frames)
FR4	Isolate one article region from a photographed page, by column projection where the frame is a close-up and by the detector otherwise, and refuse rather than guess when neither succeeds.	test_closeup (11); test_layout (30); test_api: no_single_article_asks_the_user_to_move_closer
FR5	Attach the article's own headline where one can be located by all three structural tests, and emit no title at all otherwise. A wrong headline is a failure; an absent one is not.	test_article_boundary: a_real_headline_is_found; a_masthead_far_above_the_body_is_refused; no_headline_means_no_title_not_a_wrong_one
FR6	Correct image orientation from EXIF on arrival, before any geometric processing.	test_exif (7, including agreement with an independent reference)
FR7	Judge whether recognised text is fit to be read aloud. Text that is merely short is read with a warning; text that is fragmented is replaced by a spoken explanation.	test_quality: short_and_shattered_are_different_failures; test_api: shattered_text_is_replaced_not_read_aloud; a_short_news_brief_is_still_read
FR8	Answer the nine navigation and inspection commands from the article held in session, without any network call, and without a navigation command being absorbed by the read-aloud intent.	test_local_commands (18, including from_the_start_is_not_swallowed_by_read)
FR9	Hold the article and the reading cursor in session with a bounded lifetime and a bounded capacity, with the cursor expiring together with the article it points into.	test_session (11); test_ask: the_cursor_is_per_job_not_global
FR10	Emit a downstream payload carrying the corrected text, the raw text and the changed tokens marked as diff-derived, carrying no per-token label and no confidence value.	test_rag_payload: no_confidence_and_no_label_anywhere; token_source_says_what_the_array_is
FR11	Route a genuine question to the generation component when it is enabled, and speak an intelligible sentence when that component is unavailable, empty or failing.	test_services_http: rag_down_still_speaks; an_empty_rag_answer_is_a_failure_not_silence; rag_off_says_so_instead_of_pretending
FR12	Report every failure in the same fields as a success, and give two different failures two different spoken instructions.	test_api: failure_keeps_the_same_shape; nothing_read_is_not_reported_as_success; no_single_article_asks_the_user_to_move_closer

ID	Requirement and acceptance criterion	Verified by
FR13	Guide the user to a capture distance that satisfies the glyph-height requirement and release the shutter without user action once the frame is in range, sharp and steady.	Chapter 6: eight consecutive auto-fired captures, all above the pass mark
FR14	Where optional language-model post-editing is enabled, write to a separate field, never to the body text, and reject any rewrite that is unfaithful by similarity, length or language.	test_polish (18, including the_polished_text_lives_in_its_own_field and no_evaluation_tool_can_switch_this_on)

Two requirements deserve comment because their wording encodes a decision rather than a description. FR5 states that a wrong headline is a failure while an absent one is not. Mastheads, page numbers and section strips are all headline-sized, and reading one aloud as though it were the article's title would mislead a listener who has no way to check. Silence on the title is recoverable; a confident wrong title is not.

FR10 is a requirement to withhold information. The component could emit a per-token confidence and a downstream component would be entitled to act on one, but the research supports an improvement in corrected text and does not support a per-token confidence. The payload therefore carries a diff-derived change list, labelled as such, and nothing more.

4.1.3 Non-functional requirements

Table 4.2: Non-functional requirements, acceptance criteria and status

ID	Quality attribute and criterion	Status
NFR1	Accuracy. Correction must reduce character error rate against uncorrected recognition with a 95% confidence interval excluding zero.	Met: 0.1197 to 0.0757, CI [-0.0615, -0.0199]
NFR2	Safety of correction. The proportion of already-correct word types altered by correction must be small and must be reported, not assumed.	Met and reported: 3.3% of 3,350 correct word types
NFR3	Responsiveness. The listener must receive the headline before the body, so that the perceived wait is covered by speech already in progress.	Met by design; measured stage timings on the development hardware
NFR4	Graceful degradation. No failure of a sibling component may interrupt the reading path. Each unavailable service removes one capability only.	Met: every integration default leaves the reading path unchanged
NFR5	Reliability. No condition reachable by a user may produce a stack trace or silence in place of speech.	Met: the outbound HTTP module cannot raise; scoring cannot raise; a generator crash is spoken

ID	Quality attribute and criterion	Status
NFR6	Reproducibility. A reader with the released artefacts must be able to reproduce the reported figures, and the environment under which they hold must be stated.	Met: locked split manifest, verification script, recorded version pins
NFR7	Maintainability. Every tunable constant must live in one module together with the measurement or the design decision that produced it.	Met: 400-line configuration module; no constant is hard-coded in a layer
NFR8	Interoperability. The inter-component data shape must have exactly one definition, and a unilateral change to it must fail a test rather than a demonstration.	Met: one schema module; contract tests
NFR9	Privacy and security. Photographs and recognised text must not outlive the session; credentials must not enter version control; the default reading path must require no external service.	Met: in-memory session with TTL; environment file excluded from version control; default path is local
NFR10	Accessibility. Every state of the system must be perceivable without sight, and no state may be indicated only by silence.	Met in the capture application: spoken guidance, re-announcement after six seconds, audible cue at the start of the burst
NFR11	Portability. The capture application must work on a commodity Android handset without additional hardware.	Partially met: calibrated on one handset; the calibration constant is device-specific
NFR12	Scalability. The service must support concurrent readers.	Not met: session state is in memory and single-user. Stated limitation, Section 7.5

NFR11 and NFR12 are recorded as partially met and unmet respectively. Declaring them met would be straightforward and would be false. The calibration constant that converts on-device line pitch to glyph height was fitted on one handset at one operating point, and the underlying relationship is affine rather than proportional, so it does not transfer. Session state is held in process memory with no persistence, which bounds the system at one reader at a time and discards every held article on restart. Both have known fixes that were out of scope for the submission deadline.

4.1.4 Test case scenarios

The requirements are exercised by automated scenarios in the suite rather than by manual procedures, so each can be re-run by an examiner, and Table 4.1 names the test module and function that verifies each functional requirement. The scenarios that matter are the ones distinguishing correct behaviour from behaviour that merely looks correct: a shattered page replaced by a spoken explanation rather than read aloud, a

short news brief read with a warning rather than refused, a sibling component's failure reply distinguished from a real answer, a navigation command routed ahead of the read-aloud intent, and a payload leaving the component with no confidence score attached.

Three of those scenarios exist because the failure they describe actually occurred. One was a defect found during integration: a sibling component signalled failure by returning a well-formed Sinhala sentence in the field the system speaks, which was indistinguishable from success until an explicit flag was added to the contract. Another was a routing defect caused by keyword overlap between two Sinhala intents. The third was a single failure before the readability module existed: a shattered page and a short news brief were both read aloud in the same confident voice.

4.2 System and Component Architecture

4.2.1 Position of the component within the system

The assistive reading system comprises four components built by four students against one repository: layout analysis and Sinhala recognition models, post-OCR correction which is the subject of this dissertation, retrieval and answer generation, and voice interaction with personalisation. The user-facing device is an Android handset; the processing runs on a workstation on the same network.

This component owns the path from a photographed page to corrected Sinhala text: frame selection, region isolation, the recognition call, the correction model, the readability judgement, and the session that holds the result. It also owns the capture application, because the capture requirement is a consequence of the research finding and cannot be delegated. It consumes a headline recognition model from the first component and hands its output to the third and fourth.

The architectural decision that shapes everything else is that the four components are four processes communicating over HTTP rather than four modules in one interpreter. Section 4.3.1 gives the evidence; Section 5.3 gives the implementation.

4.2.2 Internal structure of the correction path

Within the component the processing is a linear sequence of layers, each of which receives a document object, adds to it, and passes it on. The layer sequence is capture and upload, frame selection, region isolation, headline recognition, body recognition

and correction, optional post-editing, assembly, and speech. A layer may be replaced without touching the others, and the optional layers may be disabled without changing the behaviour of the rest, which is the property that lets the default configuration be exactly the configuration under which the reported numbers were measured.

The correction layer has a specific internal shape that follows from the research. Recognised text is split into sentences, each sentence is corrected independently, and the sentences are rejoined. Correction is applied to the whole sentence rather than to flagged spans within it, which is the outcome of the architectural comparison reported in Chapter 6. Sentences are decoded in batches of eight, a setting verified to produce output identical to single-sentence decoding on the full test set.

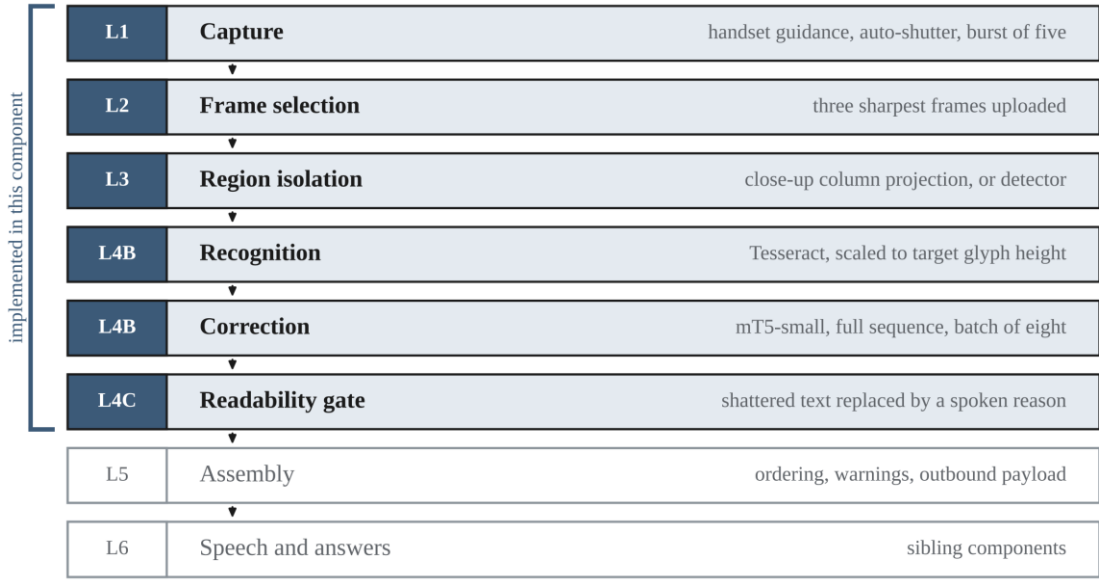


Figure 4.1: The reading path, with the layers owned by this component shaded

Two cross-cutting modules sit beside the layer sequence rather than inside it. The configuration module holds every tunable constant together with its provenance, and every layer imports from it. The schema module holds the data contract, and every layer reads and writes the document through it. These two modules are why the layer sequence can be modified safely: a constant cannot drift away from the measurement that justified it without that being visible in one file, and a field cannot be renamed on one side of a component boundary without a test failing.

4.2.3 The data contract

One module defines the data that passes between layers and between components, written as typed models. The contract is deliberately narrow: a document carries articles; an article carries a title, a body, a raw body, warnings, and diagnostic measurements; a frame carries its own quality metrics. The first component writes the title field, this component writes the body fields, and neither writes the other's. The rule is enforced by a test rather than by convention.

The payload handed to the generation component is defined in the same place and is the narrowest interface in the system: corrected text, raw text, and the tokens that differ between them, with the derivation of that token list declared.

4.3 Design Alternatives Considered

This section records the decisions where a real alternative existed, together with the evidence that settled each one. Several reversed an earlier choice, and those are the more useful entries.

4.3.1 Four processes over HTTP against one shared interpreter

Importing the sibling components as libraries would be simpler to deploy and would remove network calls from the critical path. It was rejected for two reasons, either of which alone would have been sufficient.

The requirement sets are mutually unsatisfiable. This component pins numpy 1.26.4, OpenCV 4.9.0 and transformers 5.1.0 against a CUDA 12.1 build of PyTorch; the voice component requires numpy 2.4.4 and transformers 5.7.0; the generation component requires a LangChain and ChromaDB stack; the recognition component maintains two further environments. The pin on this side is not incidental: two reported results were measured under it, including the finding that the same model checkpoint produces a character error rate of 0.0730 under one library combination and 0.0615 under another.

The second reason is operational: a component that is down should degrade one feature rather than stop the system. With four processes, a sibling that has not started removes one capability and the reading path continues. With one process, an import-time failure in any component prevents the system from starting at all, and three of the four construct expensive objects at import time.

4.3.2 Specifying capture as a glyph height against specifying a camera

An early version of the capture specification named a sensor resolution: a minimum of 13 megapixels, ideally 48. The capture-resolution sweep overturned it. Recognition accuracy against image scale is not monotonic; it has an optimum, and supplying more pixels without downscaling to that optimum measured worse, not better. Upscaling is worse again, because it cannot restore detail the camera never captured. The specification was therefore rewritten in the units that govern the outcome: a target base glyph height in the image handed to the recogniser, with a floor below which dependent vowel signs are unrecoverable.

4.3.3 Which quantity the handset measures

Having specified capture as a glyph height, the capture application must measure something on the live preview that maps onto it. Three candidates were evaluated by reproducing the whole chain on rendered pages and then degrading them, rather than by assuming a measurement accurate on clean images stays accurate on photographs.

Computing the research metric itself on device requires no calibration constant at all, and on pristine renders it is accurate to within a few percent across the operating range. Under simulated hand-held conditions it inflated by 52.3 percent, because blur fuses adjacent components and every merge enlarges a bounding box; individual components grew by up to 147 percent. Adopting it would have added a large native dependency in order to be quantitatively wrong under precisely the conditions the application operates in.

A simple ink-run height was rejected for the opposite reason: a fixed ink threshold made it insensitive, and it stayed pinned at the same value across the entire operating range. Line pitch was adopted. Blur thickens ink but does not move a baseline, so pitch drifted by less than one percent under every degradation condition, and averaging across roughly twenty-five lines gives sub-pixel resolution rather than the one-pixel steps a single run length allows. The cost is a device-specific calibration constant, recorded as NFR11.

4.3.4 The article detector against column projection

An article detector was trained and is retained in the codebase, and the original design used it to choose which story to read. It is now disabled by default on the phone path, and this is the clearest example in the project of a measurement overturning a design.

The detector was trained on full-page and half-page framings, but the capture application only fires in a band that puts the handset much closer, where the frame already contains one article. At that range the detector does not merely miss: on one capture it returned a single confident box covering the neighbouring article's headline, and the pipeline read that instead. A probe over 70 real captures compared the detector against column projection, and Section 6.4.1 reports the outcome: the column-projection crop is the better-evidenced side, with none of 57 crops spanning an article boundary after the headline split.

The decision follows from the use case rather than from the accuracy figures alone. A detector that disagrees with the alternative on two thirds of frames may not choose what is read aloud to a listener who cannot check it. The mode is retained so the comparison can be re-run, and the source records that it should be re-enabled when the measurement changes, not before.

4.4 Social, Ethical and Security Aspects of the Design

The output of this component is spoken to a person who cannot read the page it came from. That single fact changes which properties the design has to hold, and it is the origin of several requirements that would otherwise look like over-engineering.

A sighted reader using an OCR system sees that the output is garbled and discards it. A listener receives the same garbled output in the same confident synthetic voice the system uses for correct text, with no cue that anything is wrong. Silent failure is therefore the central safety concern in the design, and three mechanisms address it: the readability gate replaces shattered text with a spoken explanation instead of reading it, two different failures produce two different instructions so the user knows what to do differently, and a headline that cannot be located with confidence is omitted rather than guessed.

Privacy is addressed structurally rather than by policy. The default reading path runs entirely on the local machine, so a photographed page need not leave the network it was taken on; the components that call external services are disabled by default and are the ones whose absence costs a feature rather than the reading itself. Captured frames and recognised text live in an in-memory session with a bounded lifetime and capacity, and are not written to a database. The limitation is the other side of the same property and is stated in Section 7.5: nothing survives a restart.

Credential handling is treated as a security requirement rather than a convenience because the repository is shared with three collaborators. Keys live in an environment file excluded from version control, with a tracked example file documenting the shape, and the contract records the command that verifies the exclusion is in force. The reasoning is written down for the team: a key committed once remains in the history of every clone and fork, and deleting it in a later commit removes it from the working tree and from nothing else.

Two further positions bear on the legitimacy of the work rather than on its engineering. The corpus consists of photographs of published newspapers, used for research measurement and not redistributed; the scraped sentences used for synthetic training are publicly published news text, collected for the same purpose. Section 3.6 states the position in full. And the system is assistive rather than autonomous: it reads a page aloud and answers questions about what it read, and it is designed throughout to say that it could not read something rather than to produce a confident approximation of it.

5 IMPLEMENTATION, INTEGRATION AND TESTING

Chapter 4 described what the component was designed to do and why each design alternative was chosen or rejected. This chapter describes what was actually built, in what environment, how it joins the three sibling components into one working system, how it is tested, and how it is run. Where a constant appears in the source it is quoted with the measurement that produced it, and where a value was chosen rather than measured that is stated in the same sentence.

5.1 Development Environment and Technology Stack

The component is a Python service that loads two neural models and an OCR engine, together with an Android application that performs capture. Both halves were developed and measured on a single workstation and a single handset, and every runtime figure reported in this chapter and in Chapter 6 was obtained on that hardware. Reporting the hardware is not a formality: the batch-decoding constant of Section 5.2.5 is a property of this machine and would change on another.

5.1.1 Hardware and operating environment

Table 5.1: Development and measurement environment

Item	Value
Workstation	Windows, NVIDIA GeForce RTX 4060 (CUDA)
Python	3.12.5
Virtual environment	system/.venv
OCR engine	Tesseract 5.5.3, accessed through pytesseract
Tesseract language data	sin (body text); sin_raw (headline path)
Handset	Samsung SM-A705FN (Galaxy A70), Android 11
Handset camera streams	analysis 1440x1080 YUV; capture 3264x2448 JPEG
Service endpoint	uvicorn on 0.0.0.0:8000
Model store	models/mt5_plain (1.20 GB safetensors, 17.3 MB tokenizer)

The virtual environment was created with system site packages visible rather than isolated. This is recorded because it has a consequence: the interpreter resolves most third-party imports against the base installation, so the environment is reproducible from the recorded pins rather than from the virtual environment directory alone. The pins are enforced by documentation and by a check script, not by a lock file, which is a weakness of the implementation and is listed as such in Section 7.5.

5.1.2 Software stack and the reason the versions are pinned

The declared dependency set is short and deliberately so. Four library versions on this side of the system are pinned and recorded in the configuration module: numpy 1.26.4, OpenCV 4.9.0, transformers 5.1.0, and a CUDA 12.1 build of PyTorch.

The pin is load-bearing rather than cautious. Two results reported in this dissertation were measured under exactly that environment: the deskew reproducibility finding, and the library-version result in which the same model checkpoint produced a character error rate of 0.0730 under one library combination and 0.0615 under another. A result that is sensitive to the library version is only meaningful if the library version is stated, so the environment is treated as part of the experimental apparatus and is not upgraded casually.

The same pin is the first of the two reasons the four components do not share an interpreter, developed in Section 5.3.1. The sibling components pin numpy 2.4.4, transformers 5.7.0 and torch 2.11.0, a LangChain and ChromaDB stack, and two further virtual environments respectively. No single interpreter satisfies all four requirement sets.

A consequence of the pin shows up honestly in one place. The article-boundary measurements were taken with OpenCV 5.0.0 rather than the pinned 4.9.0. The functions used there are version-stable, unlike minAreaRect, so the measurement is reported, but it is marked in the source and re-stated here: that one figure was not obtained under the pinned environment.

5.1.3 Repository organisation and version control

The system lives in one Git repository shared by the four students. At the time of writing it holds 143 commits on the integration branch, with separate remote branches for each member. Work on this component was developed on its own branch and merged into the shared branch only when the test suite passed, which is what made it safe for three other people to pull from the same root while the correction path was being changed. The service source is organised as a thin application layer over a set of numbered processing layers and a small core of shared utilities.

The configuration module deserves the comment it carries at the top of the file: every tunable number lives in one place and layers import from it rather than hard-coding values. Each constant is accompanied by the measurement that produced it, the date of

that measurement, and the script that reproduces it; where no measurement exists the comment says so. This is the mechanism by which the engineering and research halves of the work stay joined.

5.2 Implementation of the Reading Path

5.2.1 Layer structure

The reading path is implemented as seven numbered layers with one owner each, as Figure 4.1 shows. The layer boundary is also the ownership boundary between the four components, which is what allowed four people to work in parallel against a single repository. Layer 4A writes the title field, Layer 4B writes the body field, and neither touches the other's fields. Any member can develop and test a layer in isolation against the schema, and a field renamed on one side without the other fails rather than failing silently during a demonstration.

5.2.2 Capture and frame selection

Capture is performed by an Android application written in Kotlin against CameraX 1.4.2, with Retrofit 2.11.0 and OkHttp for upload and Kotlin coroutines for the asynchronous capture cycle. The application targets API 35 with a minimum of API 26 and a Java 17 toolchain, and uses classic Android views rather than Compose because a thirty-frames-per-second measurement loop driving a text readout does not benefit from passing every update through recomposition.

The application exists because of a capture requirement the user cannot satisfy unaided. Sinhala dependent vowel signs occupy a few pixels at newsprint scale, and below a threshold glyph height the evidence for them is absent from the photograph entirely, so no amount of downstream correction can recover them. A blind user cannot judge that distance visually, and photographing first and reporting failure afterwards is the wrong interaction. The application therefore measures the frame continuously, speaks positioning guidance, and fires the shutter itself when the conditions are met.

On-device measurement does not compute the research metric directly. Connected component height, which the corpus metric uses, is accurate on clean renders but inflates by up to 52 percent under realistic hand-held blur, because blur fuses adjacent components and every merge grows the bounding box. Line pitch, the mean outlier-trimmed baseline-to-baseline spacing, drifted by under one percent under every

degradation condition tested, because blur thickens ink without moving a baseline. Pitch is therefore measured on device and converted to an estimated glyph height through a device-specific constant fitted at the acceptance threshold.

Three mechanisms keep the spoken guidance usable rather than merely correct. Hysteresis of two pixels prevents a hand held at the band boundary from alternating between contradictory instructions several times a second. Text-present gating separates a rejected measurement, where the page is in frame but this frame is noisy, from an absent page; only the second earns an instruction to point at the newspaper. A grace period of twenty frames, approximately seven tenths of a second, prevents a single blurred frame from resetting the guidance state.

The speech wrapper is called on every frame with the current state and decides internally whether to speak. It flushes rather than queues, so a stale instruction is cut off instead of arriving after the user has already moved. It enforces a minimum interval of 1.2 seconds but retries a suppressed state change on the next frame, because a version that speaks only on change drops the instruction permanently. And it re-announces a non-ready state after six seconds of silence, because to a listener silence is ambiguous between still too far and the application has stopped.

The shutter requires three conditions to hold for four consecutive frames, with any failure resetting the counter to zero rather than decrementing it: the guidance state is ready, the Laplacian sharpness is above a device-measured floor, and the glyph estimate has drifted by less than six percent across the window. The third condition is the calibration-free one, and it catches slow drift that individual frames are too sharp to reveal. A four-second cooldown follows, so one good moment yields one capture rather than a burst of near-identical ones.

Frame selection after the shutter is the part that turned out to matter most, and it was added in response to a measurement. Analysis is detached while the burst runs, so nothing observes the roughly two seconds during which the photographs are taken. Measured over three bursts, sharpness degraded across every burst, in one case from a trigger value of 4002 to frame sharpnesses of 1024, 273 and 229, because the user relaxes once they believe the capture is over. Raising the shutter threshold cannot fix this, because it gates the wrong moment. The burst therefore captures five frames and uploads the three sharpest, ranked on device by Laplacian variance, which also gives the server-side consensus stage a majority to vote on.

5.2.3 Region isolation

On arrival the server corrects EXIF orientation before anything else. CameraX writes rotation into EXIF rather than rotating pixels, and without the correction the detector receives pages turned by a quarter turn, which deskew cannot repair because deskew corrects degrees.

Which path a frame takes is decided by the measured glyph height of the frame itself. A frame above the close-up threshold is treated as already containing a single article and is cropped by column projection; otherwise the article detector is consulted. Measured over the 70 real phone captures stored in the work directory, 69 percent took the close-up and layout path, 29 percent fell through to the detector, and 3 percent were refused by layout analysis and fell back to the bounding box of all text in the frame.

The article detector is implemented, trained, and switched off by default, and that default is a measured result rather than a preference. Section 6.4.1 reports the probe that decided it. A detector that disagrees with the better-evidenced layout crop on two thirds of frames cannot be allowed to choose what is read aloud to a listener who cannot check it, so the phone path does not run it; the mode that restores the old behaviour is retained for comparison.

The headline threshold was corrected during the same measurement. On nine captures the tallest body line measured between 1.28 and 1.70 times the median line height, while the tallest headline band measured between 5.91 and 8.71 times, with nothing in between. The constant in use had been 1.6, inside the body range; it is now 3.0. Headline attachment requires three tests to pass, on vertical gap, horizontal overlap with the article's own columns, and contiguity of the row group, and refuses otherwise, because mastheads, page numbers and the section strip are all headline-sized.

5.2.4 Recognition and correction

Recognition uses Tesseract through pytesseract with the sin language data. The page segmentation mode is not a single setting, and treating it as one was a real defect. A single uniform text block mode is correct for a clean single-column region, which is what the layout path produces, while Tesseract's own column-aware layout analysis is needed when a region contains several columns, which a close-up frame can. Measured on one capture, the single-block mode spliced two columns together mid-sentence

while the column-aware mode preserved column order. The component therefore carries two recognition configurations and selects between them by path.

Scaling before recognition is specified as a target glyph height rather than a fixed factor, and this too replaced an earlier implementation after measurement. A fixed 0.4 downscale had been selected on a frame whose glyph height was 33 pixels, giving Tesseract about 13 pixels of base glyph. Applied unchanged to a frame at 22 pixels it gives 8.8 pixels, below the height at which diacritics survive, and the measured character error rate on that frame was 0.2193 against 0.0760 for the same frame scaled to 13.2 pixels: a factor of 2.9 from one constant. A six-point sweep showed a cliff below 11 pixels and a flat region from 11 to 22, so the target is 15 pixels with a hard floor at 11, a safe choice inside a flat region rather than a measured minimum.

Correction applies the fine-tuned mT5 model sentence by sentence with beam search at four beams and a maximum generated length of 128 tokens. Repeated n-gram suppression at size six is the one decoding constant with a direct measurement behind it: on identical inputs it reduced the character error rate from 0.0847 to 0.0515 by suppressing the generation runaway that sequence-to-sequence models exhibit on long lines. The optional language-model post-editing layer is disabled by default, for the reason given in Section 2.4.6.

5.2.5 Batched decoding

Sentences are decoded eight at a time rather than individually. The constant was raised on evidence, not on the grounds that batching is faster. Padded beam search can change results, which is the real risk, so the model verification script was run over the full 217-sentence locked test set at both settings under transformers 5.1.0.

Table 5.2: Batch size verification on the locked test set (n = 217)

Batch size	Exact match	CER	Wall time
1	199 / 217	0.061455	532 s
8	199 / 217	0.061455	56 s

The output is bit-identical and the run is 9.5 times faster. The constant carries an instruction in the source to re-measure if the model, the transformers version or the GPU changes, because none of the three is guaranteed to preserve the equality.

5.2.6 The readability gate

A bad photograph does not produce an error. Tesseract returns something, the correction model corrects that something, and the phone reads the result aloud in the same confident voice it uses for real news. For a listener who cannot see the page this is the most dangerous failure mode in the system, because it is indistinguishable from success.

The readability module scores recognised text on four measures with declared thresholds: the proportion of alphabetic characters that are Sinhala, below 0.85 for poor text and 0.70 for unusable text; the proportion of very short tokens, above 0.30 and 0.45; Unicode replacement characters per thousand characters, above 4.0 and 12.0; and a minimum of eight words. The important distinction it makes is between short and shattered. A six-word news brief is read with a warning; text long enough but fragmented is replaced by a spoken sentence telling the user the photograph could not be read clearly.

The same principle governs the wording of failures, and it is written into the integration contract as a rule: two different failures must not sound alike. No single article in this frame means move a little closer, which is an instruction that fixes the frame; nothing legible asks for another photograph. Both were once the same sentence, which sends nobody anywhere.

5.2.7 Session state and follow-up commands

The server holds the article it has just read in an in-memory session store, keyed by job identifier, with a time to live of 1800 seconds and a capacity of 32 jobs. The reading cursor is stored beside the document so that it expires together with the article it points into. Neither number is measured: both are design choices and are recorded as such in the source.

The common follow-up commands never leave the process. Repeat, read aloud, next, previous, first, length, title, warnings and stop are matched against Sinhala and English keyword sets and answered from the article held in session. The keyword order is load-bearing: the Sinhala phrase for from the start contains the verb for read, so the navigation intent must be tested first or every navigation command collapses into an instruction to read everything again. This path needs no network and no API key, and it is why the most frequent interaction survives an outage of the other components.

5.2.8 Configuration as a single source of measured constants

Every threshold named in this chapter lives in the configuration module and is imported by the layer that uses it. The file separates constants that answer different questions even when they look similar, which prevented a recurrence of a defect that had conflated two of them. The capture gate asks whether a photograph is good enough to recognise at all and is set at a glyph height of 25, a value that reproduces the corpus diagnostic verdict on 165 of 168 pages. The capture warning asks whether the user should move and is set lower at 20, because the whole-article framing sits at 22 and reading it there measured better. A third constant at 15 marks the point below which a frame is unusable rather than marginal; it is adopted from the handset's own guidance band and is a design choice stated as one.

5.3 Integration with the Other Components

5.3.1 Process isolation over HTTP

The four components run as four processes and communicate over HTTP. None imports another's code. Two reasons support the decision and both were decisive.

The first is the version pin of Section 5.1.2. The four requirement sets are mutually unsatisfiable in one interpreter, and importing another component's code into this process would change the environment under which the deskew and library-version results were measured, invalidating a reported number in order to avoid a network call.

The second reason matters on the day of the demonstration: a component that is down degrades one feature instead of killing the system. A sibling service that has not started, a laptop not running its local model server, or a container that failed to come up each removes one capability and leaves the reading path intact.

The entire outbound surface of this component is a single 65-line module. It performs one JSON POST with a timeout and it never raises. It uses the standard library's urllib rather than a third-party client, because in a codebase that pins library versions for a measured reason, adding a dependency for four lines of HTTP is a poor trade. Every caller is on the path to a blind user's ear, so a service that is down, a timeout, a reply that is not JSON and a reply of the wrong shape all return a failure flag and a short reason the caller can turn into a sentence.

5.3.2 The contract and the API surface

One module is the single definition of the data passing between layers, written as Pydantic models in one file and imported by every layer on this side, which states the obligation plainly: a change here breaks the other three, so it is changed here first and announced. Table 5.3 gives the HTTP surface.

Table 5.3: HTTP endpoints exposed by the component

Endpoint	Purpose
POST /capture	receive a capture burst, return title, body and warnings
POST /ask	answer a follow-up question about the article in session
GET /session/{job}	the session state for a job
GET /document/{job}	the full structured document for a job
GET /frame/{job}	the frames that were uploaded for a job
GET /audio/{job}	synthesised speech for a job
GET /latest	the most recent job, for the diagnostic console
GET /health, /health/all	liveness of this component and of its siblings
GET /, /dashboard, /debug	operator and diagnostic pages

Two properties of the capture reply are specified rather than incidental. The fields the Android application reads by name are fixed, and a failed capture still carries every one of them, because a blind user can only be told what went wrong if the explanation arrives in the fields the application already reads. Diagnostic fields such as stage timings and quality scores travel in the same reply and are ignored by the phone.

The question endpoint carries a similar rule. The phone speaks the speakable field whenever it is non-empty, whatever the success flag says: the flag records whether an answer was generated, not whether the user should hear something. Silence is the one unacceptable outcome.

5.3.3 What leaves this component

The payload handed to the retrieval and generation component carries the corrected text, the raw recognised text, and the tokens that changed between them, with the token source marked as a diff. It carries no per-token label and no confidence score, and that omission is deliberate and is asserted by two test modules.

The reported contribution of this component is an improvement in corrected text, from a character error rate of 0.1197 to 0.0757 and a word error rate of 0.3358 to 0.1640 on 217 page-disjoint test sentences, with the detector-gated span corrector as the reported negative result. Emitting a per-token confidence would put a number in front of a

downstream component that the research does not support, and a downstream component that received one would be entitled to act on it. The changed-token list is derived from a string diff, it is labelled as such, and it claims nothing about whether a change was correct.

5.4 Verification and Software Testing

5.4.1 Testing strategy

The suite is not organised around code coverage. It is organised around the four classes of mistake that this particular system makes, which were identified from the defects that actually occurred during development.

Contract tests assert that the field names and shapes crossing a component boundary are what the other side expects. They are the mechanism by which a field renamed on one side becomes a red test rather than a silent failure in front of a panel.

Provenance tests assert that a constant in the configuration module still agrees with the measurement it claims to come from: the close-up trigger must still match the handset's ready band, and the corpus verdict reproduction must still reproduce the corpus verdict. A constant that drifts away from its justification is a documentation defect the suite can catch, and Section 5.4.3 reports that one such test currently fails.

Behaviour tests assert the decisions described in Section 5.2: that a shattered page is replaced rather than read, that the post-editing layer refuses an over-large rewrite, that the payload leaving the component carries no confidence score, and that two different failures produce two different sentences.

Regression tests pin defects that have already happened once, including the indistinguishable failure reply of Section 5.3.2 and the metric-hygiene checks that guard against a character error rate being computed under a configuration that would make it unquotable.

5.4.2 Composition of the suite

The suite comprises 257 test functions across 21 modules and 3,075 lines of test code, against 5,788 lines of implementation across the application, core and layer trees: approximately one line of test for every two lines of code under test. Parametrisation expands those functions to 325 collected test cases. The largest module groups are the

service and contract tests, the capture and layout tests, and the text and metric-hygiene tests.

5.4.3 Current outcome, including the two failures

Of the 325 collected test cases, 323 pass and two fail. Both are recorded here rather than resolved silently, because both are informative and neither sits on the correction path this dissertation reports.

The first concerns the deskew stage. The second is a provenance test of the kind described in Section 5.4.1: it asserts that the server's close-up trigger still coincides with the handset's ready band. That assertion became false when the trigger was lowered from 28 to 20 on the evidence of Section 5.2.3, which showed that the whole-article framing sits at 22 and reads no worse there. The failing test is reporting exactly what it was written to report, and the correct repair is to re-pin it to the new measurement rather than restore the old constant.

Both failures are in the capture and layout layer. No test of recognition, correction, the payload contract, the readability gate or the session path fails.

5.4.4 Model verification

Beyond the unit suite, a separate script re-runs the released checkpoint over the locked test set and prints the exact-match count, the character error rate and the elapsed time. It is the script that produced Table 5.2, and it is the means by which a reader with the released artefacts can confirm that the checkpoint in the repository is the checkpoint that produced the reported numbers. Appendix A lists it among the reproducibility artefacts; it is named here because it is also the component's acceptance test for any change to the model, the decoding configuration or the inference environment.

5.5 Deployment and Operation

5.5.1 Running the system

The service is started with uvicorn bound to all interfaces on port 8000, with the project root supplied either as an environment variable or as a command line argument. Models are loaded once at startup rather than per request: loading them per request adds seconds to every capture. A launcher script starts the reader together with the sibling services, with a stub mode that runs the whole system with no API key for

development and rehearsal, and a check script that reports whether the key is present and the model endpoint reachable before anything is demonstrated.

A browser console on the service exposes every stage of the last capture, accepts uploaded frames directly, and allows questions to be asked of the article in session, because a pipeline whose intermediate stages are invisible cannot be debugged from a phone.

Operation over a wireless network proved the least reliable part of the deployment, so the system is configured to fall back to a direct USB connection between handset and workstation, which removes the network, the firewall and client isolation from the path at once.

5.5.2 Secrets and repository hygiene

Keys live in an environment file excluded from version control, with a tracked example file documenting the shape with empty values. The exclusion is a hard requirement rather than a convenience: the remote is shared with three collaborators, and a key committed once remains in the history of every clone, so rotation becomes the only remedy. The integration contract records the command that verifies the exclusion is in force, because an untracked file and an ignored file are not the same thing.

6 RESULTS AND VALIDATION

This chapter reports what was measured. Section 6.1 gives the research results on the locked test set. Section 6.2 gives the product and technical results obtained outside the locked evaluation, including the end-to-end measurements on the deployed path, where two different measurement protocols disagree and the disagreement is reported rather than resolved by choosing the favourable one. Section 6.3 covers user validation and Section 6.4 the integration measurements.

Two conventions apply throughout and are stated once. All figures in Section 6.1 are sentence-level and unnormalised, computed on the locked test set of 217 page-disjoint sentence pairs defined in Section 3.2. Figures in Section 6.2 are item-level and computed after Unicode normalisation, on different material; they are not comparable with Section 6.1 and are never placed in the same table as them.

6.1 Research Results

6.1.1 How much error there is to correct

Uncorrected recognition on the locked test set measures a character error rate of 0.1197 and a word error rate of 0.3358. The word figure is the one that matters for the application: approximately one word in three is wrong in the text that would otherwise be spoken aloud. A sighted reader recovers from that rate by looking back at the page; a listener cannot.

The error rate is also stable across the three partitions of the split, which is evidence that the page-disjoint division did not concentrate easy or hard material in one place: 0.1141 on the training pages, 0.1210 on the development pages and 0.1197 on the test pages.

6.1.2 Comparison of the five systems

Table 6.1: System comparison on the locked test set (n = 217, sentence-level, unnormalised)

System	CER	WER	CER change
B1 raw recognition, no correction	0.1197	0.3358	reference
Dictionary and edit distance	0.1193	0.3187	-0.3%
N-gram spell checker	0.1432	0.3517	+19.7%
Proposed detector-gated mT5	0.1160	0.2938	-3.1%
B3 full-sequence plain mT5	0.0757	0.1640	-36.7%

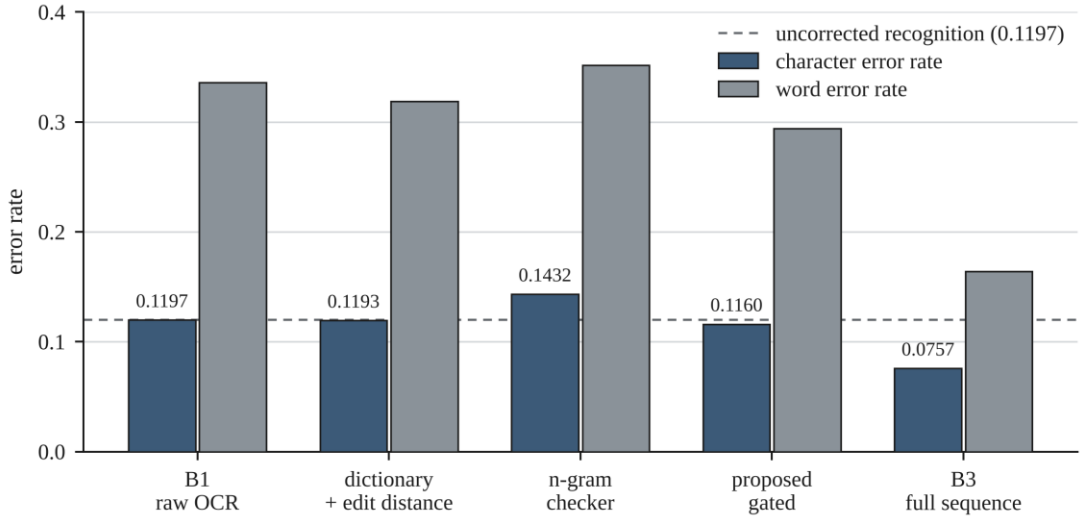


Figure 6.1: Character and word error rate for the five systems compared. The dashed line marks uncorrected recognition.

Full-sequence correction with a fine-tuned mT5-small reduces the character error rate from 0.1197 to 0.0757, a relative reduction of 36.7 percent, and the word error rate from 0.3358 to 0.1640, a relative reduction of 51.2 percent. The word-level figure is the larger because many recognition errors are confined to one or two characters inside a word: repairing those characters converts a wrong word into a right one, so the word metric moves further. For a system whose output is spoken, that is the more meaningful direction of improvement.

The two classical baselines are informative by their weakness. Dictionary lookup with edit distance leaves the character error rate essentially unchanged at 0.1193, a reduction of 0.3 percent, while improving the word error rate to 0.3187. It can only repair a word close enough to a dictionary entry and far enough from any other, and the dominant errors here are single dependent vowel signs, which frequently produce another valid word. The n-gram spell checker is worse than doing nothing, raising the character error rate to 0.1432. Neither baseline uses context beyond the word, which is precisely the information the transformer has and they do not.

6.1.3 Statistical significance

Differences between systems were tested by paired bootstrap over sentences with 10,000 resamples, reporting the percentile 95 percent confidence interval on the difference in character error rate.

Table 6.2: Paired bootstrap, 10,000 resamples, 95% CI on the CER difference

Comparison	95% confidence interval	Interpretation
B3 against raw recognition	[-0.0615, -0.0199]	significantly better; lower in 100% of resamples
Proposed against raw recognition	[-0.0062, -0.0011]	significantly better, but the effect is marginal
Proposed against B3	[+0.0163, +0.0581]	significantly worse

All three intervals exclude zero. The first establishes the primary result. The second shows that the proposed architecture does improve on doing nothing, but by an amount whose upper bound is 0.0062 character error rate, which is not a difference a listener would notice. The third is the architectural result of this dissertation, stated in the direction the evidence supports: the proposed detector-gated architecture is significantly worse than the simpler control it was designed to improve on.

6.1.4 Why gating fails, and that it is not a tuning problem

The obvious response to an architecture that underperforms is that its operating point was chosen badly. That explanation was tested directly by sweeping the detector threshold from permissive to strict across nine settings and re-running the full gated evaluation at each point.

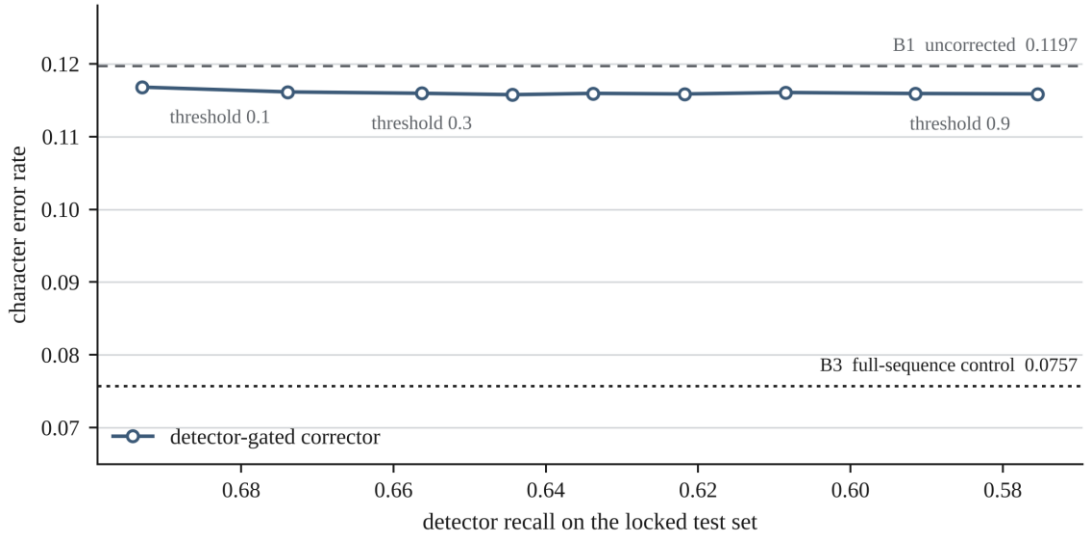


Figure 6.2: Gated character error rate against detector recall across nine thresholds. The architecture does not approach the full-sequence control at any operating point.

The gated character error rate is flat. Across the entire sweep it moves between 0.1158 and 0.1168, and the best value attainable at any threshold, 0.1158, remains far above

the 0.0757 the full-sequence system achieves. The architecture does not approach the control at any detector recall up to 0.69. Threshold selection is therefore not the explanation, and the ablation is what converts the negative result from an anecdote into a finding.

The cause is a structural ceiling. A gated corrector cannot repair an error the detector did not flag, so detector recall bounds the proportion of error operations that can possibly be fixed; the achieved fix rate against that bound is the reachability ratio defined in Section 3.4.4. The gated system repaired 299 of the 1,329 error operations in the test set, a fix rate of 0.225, against a detector recall of 0.656, giving a ratio of 0.34. The failure is not that the detector misses too much: it is that the corrector does not repair what it is given, because a span presented without its sentence context is a harder problem than the same span presented inside one.

The detector itself is adequate at its own task. At the operating threshold it achieves a recall of 0.65 and a precision of 0.95 with a false-positive rate of 1.6 percent on clean real development text, which is the behaviour the threshold selection rule of Section 3.3.4 was designed to produce. The architecture fails despite the detector working, not because of it.

6.1.5 Over-correction: the one dimension on which gating wins

Over-correction counts words that were already correct in the recognised text and were altered by the corrector. It is the harm that gating was designed to prevent, and gating prevents it.

Table 6.3: Over-correction on the locked test set

System	Correct word types altered	Rate
B3 full-sequence mT5	111 of 3,350	3.31%
Proposed detector-gated	21 of 3,350	0.63%

The gated architecture damages correct words at approximately one fifth the rate of the full-sequence system. The result is reported because it is real and because it is the honest case for the rejected design, not to rescue it; prior work treats the damage a corrector does as a first-class result rather than a footnote [36][9]. A system that preserves correct words while failing to repair incorrect ones has achieved safety by doing less. The finding does, however, define the condition under which gating would

be the right choice: an application in which an incorrect word is tolerable but a corrupted correct word is not.

6.1.6 Where the improvement comes from

The headline figure of 36.7 percent is an aggregate over error types that are not equally linguistic. Reporting it without decomposition would overstate what the model achieves on Sinhala script specifically, so the 1,329 error operations in the test set were classified by the character class involved and the fix rate computed per class. Figure 6.3 gives the per-category result and Table 6.4 the grouped one.

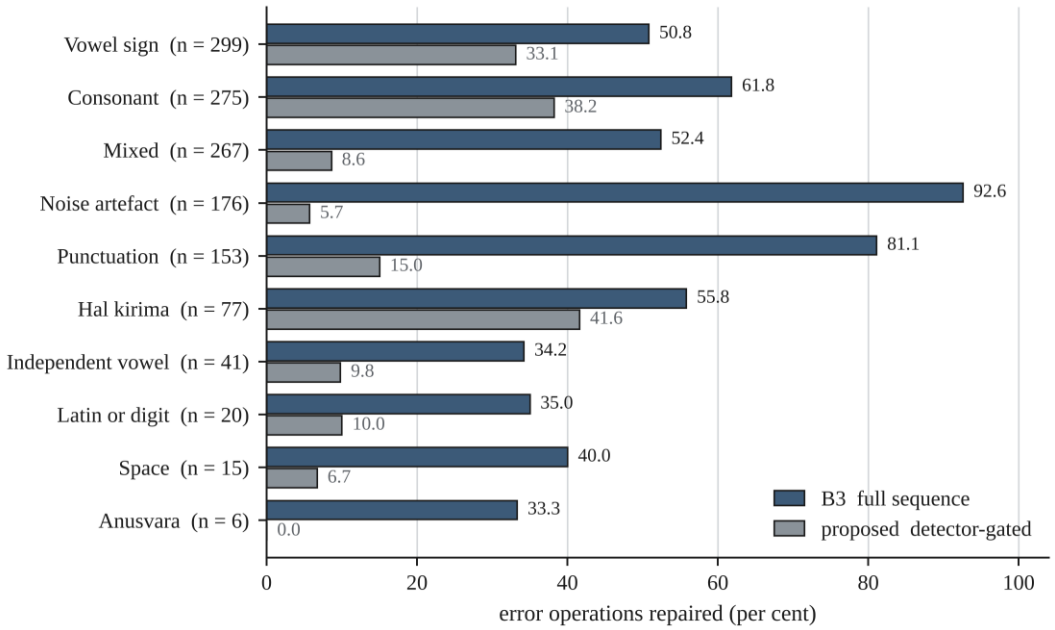


Figure 6.3: Proportion of error operations repaired, by error category

Two facts about dominance must be stated together, because stating either alone looks like selection. The single most frequent confusion in the corpus is a consonant substitution, and consonants account for 20.7 percent of all error operations. In aggregate, however, dependent vowel signs are the largest single category at 22.5 percent, and they are also the class the model repairs least well among the frequent categories, at 50.8 percent. That is consistent with the optical argument running through this dissertation: a vowel sign occupies a few pixels, and when those pixels are gone the evidence needed to choose between candidates is gone with them.

Grouping the categories separates linguistic repair from the removal of non-linguistic debris, and this is the decomposition that qualifies the headline number.

Table 6.4: Fix rate by error group

Group	n	B3 fixed	Gated fixed
Sinhala script errors	698	54.6%	34.4%
Formatting and junk	364	82.4%	9.9%
Mixed, multi-character	267	52.4%	8.6%

The model is markedly better at removing formatting debris, at 82.4 percent, than at repairing Sinhala script errors, at 54.6 percent. Both are useful to the application, since a stray underscore read aloud is as disruptive as a wrong letter, but they are different achievements. The defensible claim is therefore the narrower one: a 36.7 percent reduction in overall character error rate, of which a 54.6 percent repair rate on Sinhala script errors is the linguistic component. Chapter 7 uses the narrower figure when comparing against published work.

The grouping also exposes the gated architecture's weakness. On Sinhala script errors it reaches 34.4 percent against the control's 54.6 percent, a real but not catastrophic gap. On formatting and junk it collapses to 9.9 percent against 82.4 percent. Noise artefacts and stray punctuation are precisely the material a token-level detector trained on word-level error labels is least equipped to flag, and what it does not flag, the gated corrector never sees.

6.1.7 Data scaling and the position of the ceiling

Increasing the real photographed material from 38 pages to 230 changed the full-sequence character error rate from 0.0763 to 0.0757. A sixfold increase in real data moved the result by 0.0006.

The interpretation is that the system is near a ceiling that is optical rather than statistical. The errors that remain are predominantly cases in which recognition destroyed the information needed to recover the word, and no corrector operating on the recognised text can restore evidence that is not present in it. This is the single most actionable finding in the dissertation for anyone continuing the work: the next increment of accuracy is to be found at capture and recognition time, not in more correction training data.

6.2 Product and Technical Results

The measurements in this section were obtained outside the locked sentence-level evaluation, on different material and under a different metric protocol. They are item-level and computed after Unicode normalisation, so the absolute values are not comparable with Section 6.1; the comparisons that matter here are internal to each table.

6.2.1 The capture-resolution sweep

If the ceiling is optical, the scale at which an image is presented to the recogniser should matter, and it does. The same material was recognised at five downscale factors and scored under one metric; Figure 6.4 gives the result.

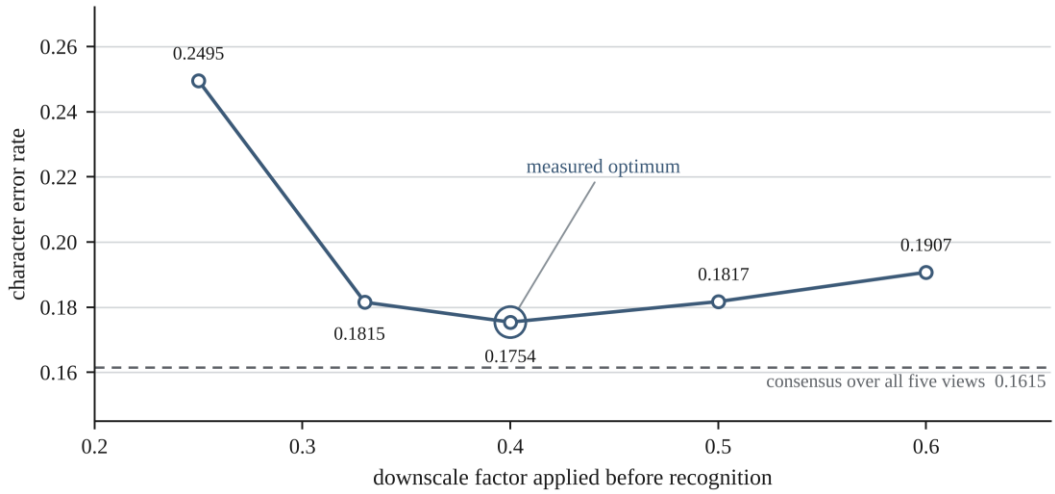


Figure 6.4: Character error rate against the downscale factor applied before recognition (item-level, normalised, uncorrected)

The relationship is not monotonic. Accuracy improves as the image is reduced from 0.60x to 0.40x, then degrades again below it, with a clear optimum at 0.40x and a steep penalty at 0.25x. Supplying the recogniser with more pixels than the optimum is not neutral; it measured worse. This is the result that overturned the earlier capture specification expressed in megapixels, described in Section 4.3.2, and replaced it with a target glyph height.

6.2.2 Multi-frame consensus

Character-level voting across the five views reduced the character error rate from 0.1754, the best single view, to 0.1615, a relative improvement of 7.9 percent, with

word error falling from 0.6213 to 0.5889. Word-level voting was less effective than character-level voting at 0.1691, which is consistent with the error profile: most errors are single characters inside otherwise correct words, so voting at word granularity discards agreement that voting at character granularity can use.

The condition under which voting helps was measured rather than assumed, and it is the more useful half of this result. Across views of comparable quality, consensus improved on the best single view by 7.9 percent. Across views of mixed quality, using identical voting code, consensus was 3.7 percent worse than the best single view, because a majority of poor views outvotes a good one. The capture design satisfies the comparable-quality condition by construction, as Section 5.2.2 describes.

6.2.3 End-to-end measurement, and a disagreement between two protocols

The two findings above and the correction model combine into a two-by-two comparison: native-scale recognition against consensus recognition, each with and without correction. The comparison was run twice under two measurement protocols, and the two runs do not agree. Both are reported.

Table 6.5: End-to-end, unaligned item-level protocol (n = 98 items)

Condition	CER	WER
A native-scale recognition	0.2345	0.7097
B native-scale recognition with correction	0.2958	0.6593
C consensus recognition at 0.40x	0.1615	0.5889
D consensus recognition with correction	0.2423	0.6024

Table 6.6: End-to-end, aligned sentence-level protocol (n = 217)

Condition	CER	WER
A native-scale recognition	0.1257	0.5016
B native-scale recognition with correction	0.0649	0.2376
C consensus recognition	0.0848	0.4055
D consensus recognition with correction	0.0365	0.2010

Under the aligned protocol the result is clean and in the expected direction: consensus improves on native scale, correction improves on no correction, and the two compose, giving a character error rate of 0.0365 when both are applied. Under the unaligned protocol correction increases the measured character error rate in both conditions, from

0.2345 to 0.2958 and from 0.1615 to 0.2423, while reducing the word error rate in both.

The difference between the protocols is the source of the disagreement and must be stated plainly rather than resolved in favour of the better-looking table. The aligned protocol uses the ground truth to select the spans compared. That treatment is identical for every condition, so the comparison between conditions is fair, but the absolute values are optimistic, because a deployed system has no ground truth with which to align. The unaligned protocol applies no such help and therefore charges every condition for coverage failures and for length differences introduced by a generative model.

The honest conclusion is that the end-to-end benefit of correction on the fully deployed path is not established by either measurement alone. The locked, sentence-level evaluation of Section 6.1 remains the result this dissertation reports, because its protocol is fixed, pre-registered and free of ground-truth assistance. The end-to-end measurements are reported as what they are: an indication that the components compose under aligned measurement, together with a warning that an unaligned measurement of the same pipeline does not reproduce it. Section 7.5 carries this forward as the principal threat to external validity.

The alignment quality of the material underlying Table 6.6 is itself reported, since the protocol depends on it: the median alignment score is 93.96 for native-scale recognition and 95.79 for consensus recognition, with no sentence falling below 60 and all 217 retained. Nothing was filtered out, which is the methodological correction described in Section 3.2.3.

6.2.4 Validation of the capture application

The capture application exists to make photographs that satisfy the glyph-height requirement. Whether it does so was measured directly, by taking eight captures with the automatic shutter after calibration and scoring them with the corpus metric.

Every one of the eight captures clears the pass mark of 25, with measured values between 29 and 34. The comparison that gives this result its force is with the research corpus itself: of the 168 photographed pages in that corpus, 7 clear the same pass mark. The corpus on which the entire study is based was photographed below the resolution its own recognition engine requires, and that is measurable from the project's own

diagnostic file rather than asserted. The capture application is the correction of that defect.

The on-device estimate reads consistently below the measured value, by three to five pixels across the eight captures. The direction is the safe one, since it makes the guidance conservative, but it confirms the limitation recorded as NFR11: the pitch-to-glyph relationship is affine rather than proportional and the single fitted constant is accurate near the acceptance threshold rather than across the range.

6.3 User and Stakeholder Validation

6.3.1 What this validation is for

Every result above is an error rate, and none of them establishes that the improvement is perceptible to the person the system is built for. A character error rate falling from 0.1197 to 0.0757 is a measurable fact about text; whether a listener understands more of a spoken article as a result is a separate question, and it is the question the application is actually answering.

6.3.2 Design of the listening study

The study compares two conditions on the same material: synthesised speech of uncorrected recognised text, and synthesised speech of the same text after correction. Participants hear a sentence once and write down what they heard. The outcome measure is word accuracy against the ground-truth sentence. Four design decisions protect the result from the bias that would otherwise make it meaningless.

- Item selection is blind to the outcome. The ten sentences were drawn by seeded random sample from the locked test set with no condition on whether correction improved them. Across the selected ten, mean character error rate falls from 0.139 to 0.021, and one of the ten is a near-null case where correction changes almost nothing; that item is retained deliberately.
- Conditions are counterbalanced across participants. Each participant hears half the items corrected and half uncorrected, and the assignment is inverted for alternate participants, so every item is heard in both conditions across the group and no participant hears the same sentence twice.
- Presentation order is shuffled per participant from a fixed seed, so that fatigue and practice effects do not accumulate on particular items.

- The participant is never told which condition an item belongs to, and the condition key is held in a separate file that is not shown during the session.

Speech is synthesised offline with a Sinhala voice at a fixed rate for every item in both conditions, so the only difference between conditions is the text. Scoring computes word accuracy per item and per participant and runs a 10,000-resample bootstrap over participants on the difference between conditions, matching the analysis used in Section 6.1.

6.3.3 Results

Six participants completed the study, each hearing ten sentences, giving 60 scored responses evenly divided between the two conditions. All six were sighted volunteers who read Sinhala; the implication of that substitution is stated in Section 6.3.5.

Table 6.7: Listening study outcome (6 participants, 60 responses)

Condition	n	Word accuracy	Clarity rating (1-5)
Uncorrected recognised text	30	0.6004	2.90
Corrected text	30	0.7921	4.17
Paired difference		+0.1917	+1.27
95% bootstrap CI on the difference		[+0.0958, +0.2882]	

Listeners recovered 60.0 percent of the words when hearing uncorrected recognised text and 79.2 percent when hearing the same sentences after correction. The paired difference, computed per participant so that the comparison is within-subject, is +0.1917, and the 95 percent confidence interval from 10,000 bootstrap resamples over participants excludes zero. Self-rated clarity moved in the same direction, from 2.90 to 4.17 on a five-point scale.

The direction of the effect is consistent across participants, which matters more than the point estimate at this sample size. Figure 6.5 shows the per-participant and per-item breakdown.

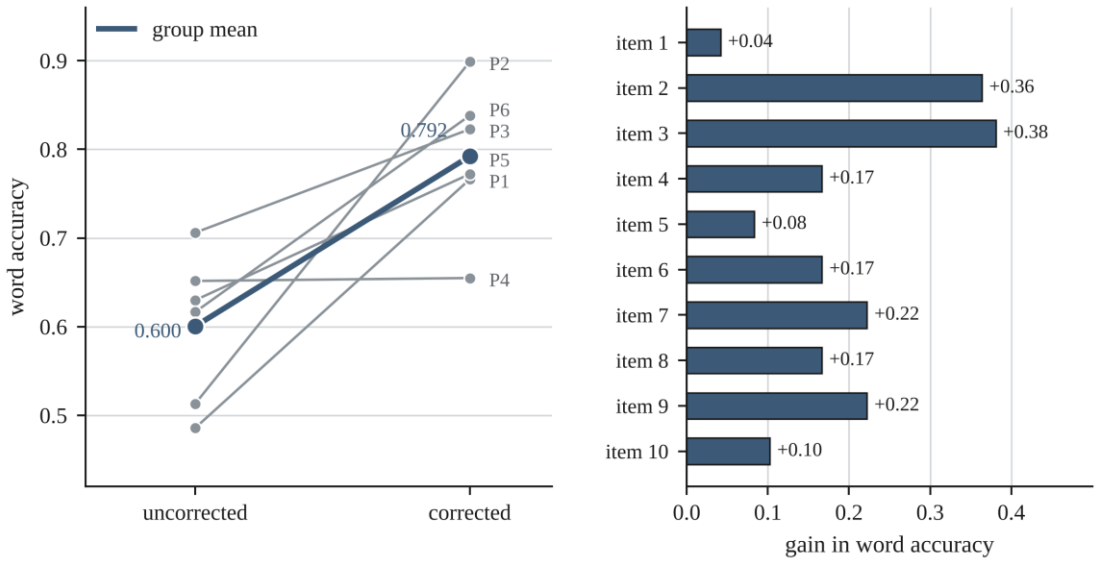


Figure 6.5: Left, word accuracy per participant in each condition, with the mean in bold. Right, the gain in word accuracy for each of the ten items.

Five of the six participants showed a clear improvement, ranging from +0.12 to +0.39. Participant 4 is effectively flat at +0.0033, and that participant is reported rather than excluded: with six participants a single flat case is the difference between a narrow interval and a wide one, and removing it would be selection on the outcome. Participant 4 also had the second-highest uncorrected accuracy of the group, which is consistent with a ceiling effect for a listener already recovering most of the words without help.

The per-item breakdown is the part of the result most capable of revealing an artefact. If the improvement were driven by one or two sentences where correction happened to work spectacularly, the aggregate would be misleading.

Every one of the ten items improved. The effect is therefore not carried by a small number of favourable sentences. Each item was heard in each condition by three different participants, so the per-item figures are means over three listeners and should be read as indicative rather than precise.

Item 5 is the near-null case described in Section 6.3.2, retained deliberately: correction moved its character error rate only from 0.0923 to 0.0769, and it produced one of the two smallest listening gains at +0.0833. That is the expected behaviour and it is evidence that the measurement is tracking the correction rather than an artefact of the two recordings.

Item 1 is the more informative case and it cuts the other way. Correction repaired it completely, from a character error rate of 0.2258 to 0.0000, yet it produced the smallest

listening gain in the set at +0.0417, because listeners were already recovering 70.8 percent of its words from the uncorrected audio. A sentence whose errors do not obstruct comprehension gains little from having them repaired. This is the clearest illustration in the study of why error rate and comprehension are separate quantities.

6.3.4 Interpretation

The research result of Section 6.1 is a reduction in word error rate from 0.3358 to 0.1640 measured on text. The result here is a different quantity measured on people: the proportion of words a listener actually recovers from spoken output rises from 0.60 to 0.79. The two are consistent in direction and in rough magnitude, which is the most this study can establish and is what it was designed to establish.

The honest reading of the clarity ratings is that they corroborate rather than evidence. Participants rated the corrected condition 1.27 points higher on a five-point scale, but a self-rating collected immediately after a transcription task is not independent of how well that transcription went. The word accuracy is the measurement; the clarity rating is a consistency check on it.

What the study does not establish is equally clear. It does not show that a listener understands the meaning of the article better, only that more individual words are recovered. Comprehension of connected discourse would require a different instrument, such as questions about the content of the passage, and Section 9.3 lists that as the natural extension.

6.3.5 Limitations of this validation

Three limitations bound this result. They were stated in the study design before the data existed, so they are not post-hoc qualification.

- The participants were sighted volunteers, not blind or low-vision readers. The study therefore measures the intelligibility of synthesised Sinhala speech under the two conditions, which is the mechanism the correction acts on, rather than the experience of the intended user. A practised blind listener is likely to recover more words in both conditions, so the absolute accuracies should not be read as predictions for that population, and the direction of the difference is the transferable finding.
- Six participants is a small sample. It bounds the precision of the estimate, which is why the confidence interval is wide at [+0.0958, +0.2882], and it is why the

analysis was specified in advance and why every participant, including the flat one, is reported.

- Ten sentences drawn from one domain. The items were sampled without conditioning on whether correction helped, which protects against the most serious bias, but they are ten sentences of newspaper text and the result does not extend beyond that.

One design property is worth restating as a strength rather than a limitation. The same speech synthesiser, at the same rate, produced both conditions for every item, so the only difference a listener encountered was the text. Whatever the shortcomings of Sinhala speech synthesis, they apply equally to both arms and cannot have produced the difference measured.

6.4 Integration Results

6.4.1 Which region-isolation method is used, and why

The decision to disable the article detector on the handset path, described in Section 4.3.4, rests on a measurement over 70 real captures rather than on a judgement. Both methods were run over the same captures and their chosen article regions compared.

Of the 51 frames on which both methods produced an answer, the two disagreed about which story was being read on 35, or 69 percent. The evidence favours column projection: none of 57 crops spanned an article boundary after the headline split. A method that disagrees with the better-evidenced alternative on two thirds of frames cannot be allowed to choose what is read aloud to a listener who cannot check it.

Measured over the same 70 captures, 69 percent took the close-up and column projection path, 29 percent fell through to the detector under the earlier configuration, and 3 percent were refused by layout analysis and fell back to the bounding box of all text in the frame. Nearly a third of real captures were therefore reaching the path the measurement above shows to be unreliable, which is why the gate was re-tuned as well as the detector disabled.

6.4.2 Headline attachment

Headline location was measured on nine real captures. The tallest body line measures between 1.28 and 1.70 times the median line height and the tallest headline band between 5.91 and 8.71 times, with no measured value in between; the threshold in use

had been 1.6, inside the body range, and is now 3.0, inside the gap. With the three structural tests applied, a headline was located and attached on eight of the nine captures, each producing recognisable Sinhala, and refused on the ninth rather than guessed.

Three limits of this measurement are recorded. Coloured headline words are lost to grayscale thresholding; a headline clipped by the frame reads partially; and nine captures from three scenes is a measured separation between two distributions, not a validated detector. The measurement was also taken under OpenCV 5.0.0 rather than the pinned 4.9.0, using functions that are version-stable.

Chapter 7 interprets these results, places them against published work, states what the dissertation contributes, and sets out the limitations that bound every claim above.

7 DISCUSSION

Chapter 6 reported what was measured. This chapter interprets it, analyses the architectural failure, places both main findings against published work, states the contributions in the narrowest form the evidence supports, and sets out the limitations that bound every claim made in this dissertation.

7.1 Interpretation of the Principal Result

Full-sequence correction reduced the character error rate on photographed Sinhala newsprint from 0.1197 to 0.0757 and the word error rate from 0.3358 to 0.1640. The second figure carries the meaning for the application. Before correction, roughly one word in three in the spoken output is wrong; after correction, roughly one in six. That is not a solved problem, and this dissertation does not claim it is; it is a halving of the rate at which a listener encounters a word that is not the word that was printed.

The asymmetry between the two metrics is informative. Word error fell by 51.2 percent while character error fell by 36.7 percent, which means the model is disproportionately repairing errors small relative to the word containing them: single dependent vowel signs, single consonant substitutions, a missing hal kirima. One character repaired converts a wrong word into a right one, and for output that is spoken rather than read, the word is the unit the listener perceives.

The classical baselines explain why a dedicated model was necessary rather than merely convenient, and the reason is a task mismatch rather than a deficiency in those tools. Spelling correctors are designed for misspelled real words and resolve them by dictionary proximity. Recognition errors in an abugida are visual confusions between characters nearly identical at newsprint scale, and a confused dependent vowel sign frequently produces another valid Sinhala word. A dictionary cannot see that such a word is wrong, because nothing about the word in isolation is wrong; only the surrounding sentence reveals it, and using the surrounding sentence is what the transformer does and the baselines do not. This is the empirical content of the phrase context-aware in the title of this work.

The data-scaling measurement is the finding with the clearest practical consequence. A sixfold increase in the scarcest and most expensive resource in the project bought almost nothing. The natural reading is that the remaining errors are not ones a corrector

can learn its way out of: they are cases where recognition destroyed the evidence, and a model operating on recognised text cannot recover information that is not in its input. The ceiling is optical, and that conclusion redirects effort away from collecting more correction data and towards capture and recognition.

7.2 Why the Proposed Architecture Underperformed

The architecture this project set out to evaluate was a detector-gated span corrector. It lost to the simpler control it was designed to improve on, and the loss was significant: the 95 percent confidence interval on the gap is [+0.0163, +0.0581], entirely on the wrong side of zero. The prior work that isolates this architecture reports that it helps [9], so this result contradicts that finding rather than confirming an expectation.

The first candidate explanation is that the operating point was chosen badly. The threshold ablation rules it out: across nine thresholds spanning recall from 0.575 to 0.693 the gated character error rate is flat, and its best value anywhere on the sweep is still far above the control's 0.0757. The failure is structural rather than a matter of tuning.

The second candidate explanation is that the detector is too weak. The reachability ratio of 0.34 rules this out as the primary cause. If the detector were the bottleneck, the corrector would be repairing most of what it was handed; instead it repairs only about a third of what it receives, so the bottleneck is downstream of the detector.

The explanation consistent with both observations is that a span presented without its sentence is a harder problem than the same span presented inside one. Correcting a confused dependent vowel sign requires knowing which word is meant, and which word is meant is determined by the words around it. Gating removes exactly that information in the act of isolating the span. The architecture protects correct words from the corrector by withholding the context that makes correction possible; the two goals are in direct tension, and in this task the context is worth more than the protection.

The per-group decomposition supports the same reading from another direction. Noise artefacts and stray punctuation are not word-level errors in the sense the token classifier was trained to recognise; they are debris sitting between words, and a token-level detector trained on word-level labels has no natural way to flag them. What is

never flagged is never corrected, so a substantial part of the gap is not a failure of correction at all but a category of error the gating mechanism structurally cannot reach.

Two further observations qualify the conclusion rather than overturning it. The gated system does achieve what it was designed to achieve on its own terms, with a safety factor of better than five against the control, and it is significantly better than no correction at all, though by an amount no listener would perceive. The honest summary is that gating buys safety by doing less, and in this application the trade is not worth making. It would be worth making where corrupting a correct word carries a much higher cost than leaving an incorrect one, and that boundary condition is a genuine output of this study.

One earlier design decision shows the same mechanism in a sharper form. An earlier version of the gated pipeline inserted explicit markers around flagged spans in the text given to the corrector. The effect was whole-sentence rewriting and fabricated content, because a marked sentence does not resemble anything in the training distribution and the model treats the whole input as material to regenerate. Removing the markers and copying unflagged words outside the model repaired that failure. Both the markers and the gating are attempts to direct a generative model by manipulating what it sees, and in both cases the manipulation cost more than it bought.

7.3 Comparison with Related Work

Absolute character error rates are not comparable across post-OCR studies, because they depend on the recognition engine, the document condition and the test material. The quantity conventionally compared is relative error reduction, and that is the quantity used here. Even so the comparison must be read with care, because a reduction measured against a weak recogniser on clean scans is not the same achievement as the same reduction measured against a stronger recogniser on photographed newsprint.

7.3.1 Against synthetic-data sequence-to-sequence post-OCR correction

The most directly comparable study is Guan and Greene [26], which applied a synthetic-data sequence-to-sequence approach across seven languages at different resource levels. The methodology resembles the one used here closely enough that the comparison is meaningful.

Table 7.1: Relative CER reduction reported for synthetic-data post-OCR correction

Language	Resource level	CER reduction
Russian	high	48.2%
English	high	39.5%
Spanish	high	37.3%
Sinhala (this work)	low	36.7%
Frisian	low	31.1%
Telugu	low	25.9%
Icelandic	low	17.9%
Irish	low	12.4%

The result obtained here is fourth of the eight entries and the highest of the low-resource languages, but it sits below all three high-resource languages. An earlier draft of this work stated that the result exceeds reported low-resource reductions and listed only the four languages below it. That framing is a selection and it is corrected here: the full list is given, and the position in it is stated without rounding in either direction.

Telugu is the comparison that carries the most information, at 25.9 percent. Telugu is a fellow abugida, so it presents the same structural difficulty: a dense inventory of dependent vowel signs rendered as small marks attached to base consonants, which is the error class that dominates this corpus. That the Sinhala result exceeds the Telugu one is encouraging, but the two were measured on different material with different recognisers and the gap should not be read as a ranking of methods.

A further qualification applies to the comparison as a whole. The 36.7 percent figure includes the removal of non-linguistic debris, which is an easier problem than script repair. Against the narrower 54.6 percent script-only fix rate the comparison is not directly available, because the published figures are aggregate reductions and are not decomposed by error class.

7.3.2 Against existing Sinhala correction work

Prior Sinhala OCR correction exists and this work does not claim to be the first. Rule-based Sinhala post-OCR correction has been reported previously [7], and its published error list includes confusions that recur in this corpus, among them the consonant confusion that is the single most frequent substitution measured here. Sinhala spelling correction by n-gram methods also predates this work [6]; a representative implementation of that family was reproduced as a baseline and is the one that made the text worse.

The claim this work can defend is narrower: a transformer-based, context-aware post-OCR corrector for Sinhala, evaluated on real photographed newsprint under a locked page-disjoint split, together with the first frequency-weighted error taxonomy derived from real photographs rather than from synthetic or scanned material.

7.4 Contributions

The contributions are stated in the narrowest form the evidence supports. Each is followed by what would be needed to make a stronger claim, so that the boundary is explicit.

A frequency-weighted Sinhala OCR error taxonomy from real photographs. 192 distinct character confusions with measured frequencies, derived from real photographed newspaper pages rather than from synthetic corruption or clean scans, together with the deletion and insertion profiles. The taxonomy is what made the synthetic training corpus realistic. Prior Sinhala work has published error lists [7]; what is new is that this one is frequency-weighted and derived from photographs of the document condition the application actually faces. A stronger claim would require a second recognition engine and a second document domain.

A context-aware post-OCR corrector for Sinhala, honestly evaluated. A fine-tuned mT5-small reduces character error by 36.7 percent and word error by 51.2 percent on real photographed Sinhala newsprint, measured on a test set locked before any model was trained, split at photograph level, and never used for model selection or threshold tuning. The evaluation design is as much a part of this contribution as the number, because a post-OCR result on a small corpus without those protections is not interpretable. A stronger claim would require more than one newspaper domain and more than one recogniser.

An architectural negative result with its cause identified. Detector-gated span correction significantly underperforms full-sequence correction on this task, the result holds across the full range of detector thresholds, and the cause is localised to the corrector rather than the detector by the reachability ratio. The study was pre-registered with a win condition that was not met, which is what distinguishes this from an unpublished failed experiment. The boundary condition under which the rejected design would be preferable is also identified: an application in which damaging a correct word costs much more than failing to repair an incorrect one.

A methodological finding about post-OCR benchmarking. An earlier iteration of this project filtered its evaluation set by alignment similarity, discarding pairs whose recognised and reference text disagreed too much. That gate removed roughly 57 percent of the data, and it removed it non-randomly: the sentences it discarded were the hardest ones, because a sentence is hard precisely when recognition and reference disagree. The resulting benchmark was inflated by roughly a factor of four to five. The project's own records give two different values for that inflated figure and the discrepancy is noted rather than silently averaged.

The corrected practice adopted in this work is to retain every pair and report the distribution of alignment quality instead of filtering on it, as Section 6.2.3 does. The general form of the finding is that similarity-based filtering of a post-OCR evaluation set is not a hygiene step; it is a selection on the dependent variable, and any benchmark that applies one is reporting performance on the subset of the data where the problem was already easy.

Capture specified as evidence rather than as hardware. The optical ceiling finding was turned into a deployment requirement: capture is specified as a target base glyph height in the image given to the recogniser, not as a camera specification, and it is enforced at capture time by an on-device estimator chosen for invariance to motion blur. The supporting negative result is that computing the research metric on the handset inflates by 52.3 percent under realistic hand-held conditions while line pitch drifts by under one percent. The measurement that makes this concrete is that 7 of the 168 pages in the project's own corpus meet the pass mark, while every automatically fired capture does.

7.5 Limitations and Threats to Validity

The limitations below are grouped by what they threaten. Several of them are among the more useful outputs of the work, because they define what a continuation should do first.

7.5.1 Threats to internal validity

- Corpus source imbalance. The synthetic training text is drawn overwhelmingly from one news source, approximately 93 percent, while the photographed test pages come predominantly from two others. The model generalised across that

gap, but the imbalance means the training distribution is narrower than the corpus description suggests.

- Four of the ten error categories contain fewer than 50 operations. Their fix rates appear in Figure 6.3 for completeness and should not be interpreted; the aggregate and the three large categories carry the result.
- The aligned end-to-end measurement uses ground truth to select the spans compared. The treatment is identical across conditions so the comparison between them is fair, but the absolute values are optimistic and a deployed system has no such assistance.
- Two metric unit systems coexist in this dissertation: the locked evaluation is unnormalised and sentence-level, while the capture-side measurements are Unicode-normalised and item-level. Every table states which applies and no table mixes them, but a reader who compares a figure in Section 6.1 with one in Section 6.2 will reach a wrong conclusion.
- Two glyph metrics coexist for the same reason: a capture gate applied to a whole frame and a resize target applied to a single crop. They were conflated under one constant earlier in the project, and the configuration module records the distinction explicitly so that it is not lost again.

7.5.2 Threats to external validity

- One recognition engine. Every error measured in this work comes from Tesseract with the Sinhala language data, and the correction model is trained on noise drawn from that engine's measured confusion distribution.
- One document domain. All material is Sinhala newspaper text. Books, forms, signage and handwriting present different layouts, typefaces and error profiles.

7.5.3 Threats to the claim of usefulness

- The user validation reported in Section 6.3 rests on six sighted volunteers rather than blind or low-vision readers. It therefore measures the intelligibility of synthesised Sinhala speech under the two conditions, which is the mechanism correction acts on, rather than the experience of the intended user, and its confidence interval is correspondingly wide. The system as implemented also holds one reader at a time in process memory and is a research prototype rather than a deployable service.

8 COMMERCIALISATION AND SUSTAINABILITY

This chapter addresses whether the work has a route beyond the dissertation. Section 8.1 states the value proposition and identifies the users. Section 8.2 sizes the need from published national statistics and positions the work against what is already available to a Sinhala reader who cannot read print. Section 8.3 proposes a sustainability model and argues why it is not a conventional commercial one. Section 8.4 covers scalability, intellectual property and regulatory compliance.

8.1 Value Proposition and Target Users

8.1.1 The problem in commercial terms

A person who cannot read print and whose language is Sinhala has very limited access to printed Sinhala text. The specific gap this work addresses is that the generic solution, photograph the page and have a phone read it aloud, does not work well enough in Sinhala to be relied on: roughly one word in three in the recognised text is wrong. A sighted user of the same tool glances at the screen, sees nonsense and discards it. A blind user hears it in a confident synthetic voice with no indication that anything is wrong, which is worse than no answer.

The value proposition is therefore narrow and specific: the component halves the rate at which a Sinhala listener encounters a wrong word, from roughly one in three to roughly one in six, and the surrounding system refuses to read aloud text it judges unreadable rather than speaking it anyway. The second property is as commercially relevant as the first, because trust determines whether an assistive tool is used twice.

8.1.2 Who the users are

Three groups are distinguished because they want different things from the same technology.

- Primary users: Sinhala-reading adults with blindness or severe low vision who read print before losing vision, and who therefore have both the motivation and the literacy to consume newspaper text aurally. This group values accuracy and trustworthiness over features.

- Secondary users: readers with low vision, dyslexia, or age-related reading difficulty who can see the page but find sustained reading difficult. For this group the same pipeline is a reading aid rather than a substitute for sight.
- Institutional users: schools for the visually impaired, libraries, the national council and non-governmental organisations serving blind readers, and public services that issue Sinhala documents. This group does not buy for itself; it deploys on behalf of the first group and is the realistic route to reach them.

The distinction matters because the primary users are the least able to pay and the institutional users are the ones with budgets. Any model that charges the person who needs it most will not reach them, which is the central constraint in Section 8.3.

8.2 Market and Competitive Position

8.2.1 Size of the need, from published statistics

Two independent national sources describe the population concerned, and they measure different things, so both are given.

Table 8.1: Published figures describing visual difficulty in Sri Lanka

Source	Measure	Figure
Census of Population and Housing 2012, Department of Census and Statistics [38]	Persons aged 5 and over reporting difficulty in seeing	996,939 (54 per 1,000)
Census of Population and Housing 2012 [38]	Persons reporting any disability	1,617,924 (87 per 1,000)
Sri Lanka National Blindness, Visual Impairment and Disability Survey, 2014-15 [39]	Prevalence of blindness, population aged 40 and over	1.7%
National Blindness Survey, 2014-15	Severe visual impairment	1.6%
National Blindness Survey, 2014-15	Visual impairment	15.4%
National Blindness Survey, 2014	Cataract as a cause of blindness	approx. 67%

The census figure counts self-reported difficulty at any severity, so it includes people for whom spectacles are the answer. The survey figure is clinically assessed but is restricted to the population aged 40 and over. Neither is the addressable group for this work, and no published figure is.

The addressable group can be narrowed only by stated assumption. It consists of people who read Sinhala, who have lost enough vision that print is inaccessible, and who have access to an Android handset. Taking the clinically assessed blindness and

severe visual impairment prevalences together, 3.3 percent of the population aged 40 and over, the group is in the order of a hundred thousand adults, and the share who read Sinhala and have handset access is smaller again. The relevant conclusion is an order of magnitude: large enough to justify sustained effort and far too small to attract commercial investment from outside the country.

Two further statistics bear on deliverability. The Department of Census and Statistics [37] reports digital literacy at 70.8 percent and computer literacy at 38.4 percent for the population aged 5 to 69 in the first half of 2025, while household computer ownership stands at 21.4 percent and 79.7 percent of internet users connect by smartphone. Any assistive tool for this population must therefore be a phone application rather than desktop software, which is what was built.

8.2.2 What a Sinhala reader can use today

The competitive position is better described as an absence of competition than as a crowded field.

Table 8.2: Alternatives available to a Sinhala reader who cannot read print

Alternative	Position for Sinhala
Dedicated wearable reading devices	Technically capable but listed from approximately US$4,250; Sinhala support is not offered. Out of reach on both counts.
On-device phone text recognition (Google ML Kit)	Sinhala is not supported at all. The supported scripts are Latin, Chinese, Devanagari, Japanese, Korean and Cyrillic [2].
Cloud text recognition (Google Cloud Vision)	Sinhala is listed as an experimental language [3], and the service requires network connectivity and sends the photographed page off the device.
Open-source recognition (Tesseract) plus a screen reader	Possible, and it is the baseline measured in this work: character error 0.1197, word error 0.3358 on photographed newsprint.
Human readers, audio books, radio	Effective but not on demand, and dependent on another person's time.

The two entries that matter are the middle ones. The dominant on-device text recognition library on Android does not support Sinhala in any form, and the corresponding cloud service classifies it as experimental. A Sinhala reader therefore cannot simply use the tool that an English, Hindi or Japanese reader uses, because that tool does not read their script. The open-source path exists, which is why this work takes it as the baseline rather than claiming that nothing was possible before.

The defensible competitive claim is therefore not that this system outperforms a commercial product. It is that for photographed Sinhala newsprint there is no commercial product, the free on-device option does not cover the script, the cloud option is experimental and requires sending the page to a server, and the open-source option produces one wrong word in three until a correction stage is added.

8.3 Sustainability Model

8.3.1 Why a conventional commercial model does not fit

The people who need this most have the least ability to pay, the group is too small to attract outside investment, and the social value of the work is realised only if the tool is in the hands of blind readers rather than held behind a price. A subscription aimed at primary users would fail on all three counts. The model proposed is therefore a sustainability model rather than a revenue model, with the explicit goal of keeping the artefact alive and improving after the dissertation is submitted.

8.3.2 The proposed model

- Free to the primary user, permanently. The handset application and the correction model are released at no cost to individuals. This is a condition of the model, not a launch promotion.
- Open release of the research artefacts. The error taxonomy, the locked split manifest, the evaluation scripts and the trained correction model are released so the work can be continued by others. For a language group this size, the realistic path to a better Sinhala reader is other researchers building on a public baseline, not a single team maintaining a product.
- Institutional deployment and support as the funded activity. Schools for the visually impaired, libraries and public bodies that wish to deploy the system at scale, integrate it with their own document stores, or have it maintained against new handsets and new recognition engines, are the parties with budgets and with a reason to pay for support rather than for the software.

8.3.3 Cost structure

The cost structure is unusually favourable, and this is a direct consequence of architectural decisions reported in earlier chapters rather than a separate claim.

The system requires no new hardware: it runs on a commodity Android handset the user or the institution already owns. The default reading path requires no external paid service, because recognition, correction and the common interactions all run locally, and the components that call external services are disabled by default, as Section 5.3.1 records. The correction model is small enough to be served from a modest workstation rather than from rented accelerators. Per-user marginal cost is therefore close to zero, and the recurring cost is maintenance effort rather than infrastructure.

The one cost that does not fall is corpus work. Every improvement in a new document domain requires photographs and manual transcription, which is human time that scales linearly. Any plan for this work that does not budget for transcription is not a plan.

8.3.4 Route to the user

The realistic route is institutional rather than direct. Organisations serving blind readers in Sri Lanka already have the relationships, the trust and the training capacity that a student project does not. The sequence proposed is: validate with a small group of blind and low-vision readers; place the application with one institution and support it properly; use the resulting failure reports to improve the capture guidance and the readability gate, which are the two components most likely to break on real users; and only then widen distribution.

8.4 Scalability, Intellectual Property and Compliance

8.4.1 Technical scalability

The honest position is that the research is scalable and the current implementation is not. Session state is held in process memory with a bounded capacity and no persistence, which means one reader at a time and nothing surviving a restart; this is recorded as an unmet requirement in Table 4.2 and as a stated limitation in Section 7.5. Converting the prototype into a service for many concurrent users requires persistent session storage and a request queue in front of the model, both of which are ordinary engineering rather than research.

Two properties do scale well as they stand. The component is already a separate process behind an HTTP contract, so it can be replicated horizontally without any change to the handset application. And because correction decodes sentences in

batches of eight with verified identical output, throughput improves roughly ninefold over single-sentence decoding on the same hardware.

8.4.2 Scaling to other languages and domains

The method transfers more readily than the model. Reproducing this work for another abugida requires photographs with ground truth, a measured confusion table derived from them, scraped clean text in the target language, and the same training and evaluation procedure. None of those steps is language-specific in design; all of them are language-specific in data. The capture specification transfers unchanged, because a target glyph height is a statement about evidence rather than about a script.

8.4.3 Compliance and responsible deployment

Three compliance considerations apply to a system that photographs documents and speaks their contents.

Privacy is addressed structurally rather than by policy, as Section 4.4 describes: the default path processes the photograph locally, captured frames and recognised text live in a session with a bounded lifetime, and nothing is written to a database. A user photographing a letter, a prescription or a bank statement is a realistic scenario, and the design assumption is that such an image must not leave the user's own network by default.

Accuracy disclosure is an ethical requirement for an assistive product rather than a legal one. The measured word error rate after correction is approximately one word in six, and a deployment must not present its output as a faithful reading. The readability gate and the warning mechanism are the technical expression of that obligation, and any product documentation should carry the measured figure rather than an adjective.

The system must not be used where a misread word carries legal or clinical consequence, such as medication labels, dosage instructions or contractual documents. This is a stated limitation of scope and it follows from the measured error rate rather than from caution.

9 CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

This dissertation asked whether context-aware post-OCR correction can reduce the error rate of recognised Sinhala newspaper text, photographed under the conditions a blind user would actually produce, by enough to change what a listener hears, and whether a detector-gated architecture is the right way to do it. The first question is answered in the affirmative and the second in the negative, and both answers are supported by measurement on a test set that was locked before any model was trained.

A fine-tuned mT5-small corrector reduced the character error rate on 217 page-disjoint sentences of real photographed Sinhala newsprint from 0.1197 to 0.0757, a relative reduction of 36.7 percent, and the word error rate from 0.3358 to 0.1640, a reduction of 51.2 percent. The 95 percent confidence interval on the character error difference is [-0.0615, -0.0199] and the direction held in every one of 10,000 bootstrap resamples. The listener goes from roughly one wrong word in three to roughly one in six. Two classical baselines reproduced on the same data did not approach this, so the improvement comes from using sentence context.

The proposed detector-gated architecture was pre-registered with a win condition it did not meet. It reached a character error rate of 0.1160 against the full-sequence control's 0.0757, and the confidence interval on the gap, [+0.0163, +0.0581], places the difference firmly on the wrong side of zero. Two explanations were tested and rejected: a nine-point threshold ablation showed the gated error rate flat between 0.1158 and 0.1168, so the operating point is not the cause, and the reachability ratio of 0.34 locates the bottleneck downstream of the detector. The architecture withholds the sentence context that makes correction possible in order to protect correct words, and in this task the context is worth more than the protection. The protection is real, at 0.63 percent of already-correct word types altered against the control's 3.31 percent, and it defines the class of application in which gating would be the right choice.

Three further findings are of comparable practical value. Increasing real photographed material from 38 pages to 230 moved the character error rate by 0.0006, placing the remaining ceiling at the optics rather than in the quantity of correction data. Recognition accuracy against image scale is not monotonic, so supplying more pixels without downscaling to the measured optimum measured worse rather than better. And

the project's own research corpus was photographed below the resolution its own recognition engine requires, with 7 of 168 pages meeting the pass mark, a defect the capture application corrects on every automatically fired capture tested.

The work leaves four artefacts: a frequency-weighted Sinhala error taxonomy of 192 character confusions measured from real photographs; a locked, page-disjoint evaluation set with its manifest and scoring code; a trained correction model with a verification script that reproduces the reported figures; and a working integrated system in which the component runs behind a documented contract with three sibling components. For a language with no shared post-OCR benchmark, the second of those may outlast the first.

A listening study closes part of the distance between the error rate and the person. Six participants transcribed ten sentences each, half in each condition, with items sampled without regard to whether correction helped them. Listeners recovered 60.0 percent of the words from uncorrected speech and 79.2 percent after correction, a paired difference of +0.1917 with a 95 percent interval of [+0.0958, +0.2882], and every one of the ten items improved.

The honest boundaries are stated as plainly as the results. The work uses one recognition engine, one document domain and one handset. The end-to-end benefit on the fully deployed path is protocol-dependent and is not established by either of the two measurements made. The listening study rests on six sighted volunteers and measures words recovered rather than meaning understood. The system as implemented holds one reader at a time in memory and is a research prototype rather than a service. Each of these limitations names something a continuation should do.

9.2 Achievement of the Objectives

The project was registered with a sub-objective to generate noisy and clean sentence pairs by adding realistic recognition errors, train a transformer model on them, and evaluate with character and word error rate using a few real recognition examples. Table 9.1 states what was done against each element.

Table 9.1: Registered objectives and outcomes

Registered objective	Outcome
Generate noisy and clean pairs by adding realistic OCR-style errors	Achieved, and strengthened: the noise model is a frequency-weighted confusion table measured from

Registered objective	Outcome
	265 real photographed pairs rather than a hand-written rule set
Train a transformer model to correct recognised Sinhala	Achieved: mT5-small fine-tuned on the synthetic corpus, reducing CER by 36.7% and WER by 51.2%
Evaluate using CER and WER with a few real OCR examples	Achieved, and substantially strengthened: evaluated on 217 real photographed pairs under a locked page-disjoint split, with paired bootstrap confidence intervals
Demonstrate context-aware correction rather than rules alone	Achieved: two classical non-contextual baselines reproduced on the same test set, one giving no improvement and one degrading the text
Evaluate the proposed detector-gated architecture	Achieved as a pre-registered comparison; the architecture did not meet its win condition and the cause was identified
Integrate the component into the group system	Achieved: HTTP contract, 325 collected tests with 323 passing, every integration default leaving the reading path unchanged
Validate with users	Achieved: 6 participants, 60 responses. Word accuracy 0.60 to 0.79, paired difference +0.1917, 95% CI [+0.0958, +0.2882] (Section 6.3)

One divergence from the registration is deliberate. The registered plan proposed evaluation with a few real recognition examples; the study was instead built on 217 real photographed pairs under a locked page-disjoint split with bootstrap significance testing. That is a strengthening of the registered plan, and it is what makes the negative result reportable, but it is a change and it is declared as one.

9.3 Future Directions

The directions below are ordered by the ratio of expected value to effort, which is not the order in which they are most interesting.

9.3.1 Immediate: close the gaps this work leaves open

- Repeat the listening study with blind and low-vision Sinhala readers. The instrument is built and validated, the analysis is pre-specified, and the result from six sighted volunteers gives an effect size to power it against. This is the single highest-value remaining task, because the population the system exists for is the one it has not yet been measured on.
- Measure comprehension rather than transcription. The study shows that more words reach the listener; whether they follow the article better needs an instrument that asks about the content of the passage rather than asking them to write it down.

9.3.2 Near term: test the boundaries of the result

- Repeat the evaluation with a stronger recognition engine. The correction model is trained on a confusion distribution measured from one engine, and the most important open question about it is whether the learned corrections transfer to a different error distribution or are specific to the one they were measured on. The locked split makes this a clean experiment.
- Compare a byte-level model against the subword model used here. Byte-level pretrained models are reported to be more robust to noise and require no language-specific tokenisation, which is an attractive property for a script whose subword segmentation is poorly supported, and the comparison is a direct substitution into an existing pipeline.
- Extend the corpus to a second document domain. Books, forms and public signage have different layouts, typefaces and error profiles, and a second domain is what would turn a claim about Sinhala newsprint into a claim about Sinhala print.

9.3.3 Longer term: the directions the findings point to

The data-scaling result argues that the next increment of accuracy is optical rather than linguistic. Two consequences follow. Effort spent on capture, on lighting, framing, stability and scale, is likely to be repaid more than effort spent on more correction training data. And a correction model with access to the image, rather than only to the recognised text, would have evidence this one does not: a model that can see that two candidate characters differ by a mark of three pixels is better placed than one that sees only the recogniser's guess. Work that injects glyph similarity on the model side is the natural starting point.

The architectural result points somewhere less obvious. Gating failed because isolating a span removes the context that makes it correctable, and markers failed earlier for a related reason: both are attempts to steer a generative model by manipulating what it sees. An approach that gives the model the whole sentence and constrains its output instead, so that it may not alter spans the detector considers sound, would preserve the context and still provide the protection. That is a decoding-time constraint rather than an input-time one, and it is the version of this architecture the evidence does not rule out.

Finally, the method is more portable than the model. Reproducing this work for another abugida requires photographs with ground truth, a measured confusion table, scraped clean text and the same training and evaluation procedure; none of those steps is language-specific in design. The capture specification transfers unchanged, because a target glyph height is a statement about evidence rather than about a script.

REFERENCES

[1] B. Gamage, T.-T. Do, N. S. C. Price, A. Lowery, and K. Marriott, "What do blind and low-vision people really want from assistive smart devices? Comparison of the literature with a focus study," in Proc. 25th Int. ACM SIGACCESS Conf. on Computers and Accessibility (ASSETS '23), New York, NY, USA, 2023, art. 30, 21 pp. doi: 10.1145/3597638.3608955.

[2] Google, "Text recognition v2: Supported languages," ML Kit documentation. [Online]. Available: https://developers.google.com/ml-kit/vision/text-recognition/v2/languages

[3] Google, "OCR language support," Cloud Vision API documentation. [Online]. Available: https://cloud.google.com/vision/docs/languages

[4] C. Granquist, S. Y. Sun, S. R. Montezuma, T. M. Tran, R. Gage, and G. E. Legge, "Evaluation and comparison of artificial intelligence vision aids: Orcam MyEye 1 and Seeing AI," Journal of Visual Impairment & Blindness, vol. 115, no. 4, pp. 277–285, 2021. doi: 10.1177/0145482X211027492.

[5] N. Sankaran and C. V. Jawahar, "Error detection in highly inflectional languages," Tech. Rep., Centre for Visual Information Technology, IIIT Hyderabad, India, 2013.

[6] A. Wasala, R. Weerasinghe, R. Pushpananda, C. Liyanage, and E. Jayalatharachchi, "A data-driven approach to checking and correcting spelling errors in Sinhala," International Journal on Advances in ICT for Emerging Regions (ICTer), vol. 3, no. 1, pp. 11–24, 2010. doi: 10.4038/icter.v3i1.2844.

[7] B. P. K. M. Balasooriya, "Improving and measuring OCR accuracy for Sinhala with Tesseract OCR engine," M.Sc. dissertation, University of Colombo School of Computing, Colombo, Sri Lanka, 2020.

[8] A. Dilhara and N. Jayatilleke, "Cross-temporal Sinhala OCR: Page-level adaptation and diachronic analysis," arXiv:2606.29378, 2026.

[9] R. Schaefer and C. Neudecker, "A two-step approach for automatic OCR post-correction," in Proc. 4th Joint SIGHUM Workshop on Computational Linguistics for Cultural Heritage, Social Sciences, Humanities and Literature (LaTeCH-CLfL), 2020, pp. 52–57.

[10] R. Smith, "An overview of the Tesseract OCR engine," in Proc. 9th Int. Conf. on Document Analysis and Recognition (ICDAR 2007), vol. 2, Curitiba, Brazil, 2007, pp. 629–633. doi: 10.1109/ICDAR.2007.4376991.

[11] World Health Organization, World Report on Vision. Geneva, Switzerland: World Health Organization, 2019, 180 pp. ISBN 978-92-4-151657-0.

[12] M. Cutter and R. Manduchi, “Improving the accessibility of mobile OCR apps via interactive modalities,” ACM Transactions on Accessible Computing, vol. 10, no. 4, art. 11, pp. 1–27, 2017. doi: 10.1145/3075300.

[13] M. Vázquez and A. Steinfeld, “An assisted photography framework to help visually impaired users properly aim a camera,” ACM Transactions on Computer-Human Interaction, vol. 21, no. 5, pp. 1–29, 2014. doi: 10.1145/2651380.

[14] L. Nanayakkara, C. Liyanage, P. T. Viswakula, T. Nagungodage, R. Pushpananda, and R. Weerasinghe, “A human quality text to speech system for Sinhala,” in Proc. 6th Workshop on Spoken Language Technologies for Under-Resourced Languages (SLTU 2018), 2018, pp. 157–161. doi: 10.21437/SLTU.2018-33.

[15] S. Kafle and M. Huenerfauth, “Predicting the understandability of imperfect English captions for people who are deaf or hard of hearing,” ACM Transactions on Accessible Computing, vol. 12, no. 2, art. 7, pp. 1–32, 2019. doi: 10.1145/3325862.

[16] N. de Silva, “Survey on publicly available Sinhala natural language processing tools and research,” arXiv:1906.02358.

[17] R. Smith, D. Antonova, and D.-S. Lee, “Adapting the Tesseract open source OCR engine for multilingual OCR,” in Proc. Int. Workshop on Multilingual OCR (MOCR ’09), 2009, pp. 1–8. doi: 10.1145/1577802.1577804.

[18] N. Jayatilleke and N. de Silva, “Zero-shot OCR accuracy of low-resourced languages: A comparative analysis on Sinhala and Tamil,” in Proc. Recent Advances in Natural Language Processing (RANLP 2025), Varna, Bulgaria, Sep. 2025, pp. 471–480. arXiv:2507.18264.

[19] T. T. H. Nguyen, A. Jatowt, M. Coustaty, and A. Doucet, “Survey of post-OCR processing approaches,” ACM Computing Surveys, vol. 54, no. 6, pp. 1–37, 2021. doi: 10.1145/3453476.

[20] C. Rigaud, A. Doucet, M. Coustaty, and J.-P. Moreux, “ICDAR 2019 competition on post-OCR text correction,” in Proc. 2019 Int. Conf. on Document Analysis and Recognition (ICDAR), 2019, pp. 1588–1593. doi: 10.1109/ICDAR.2019.00255.

[21] O. Kolak, W. Byrne, and P. Resnik, “A generative probabilistic OCR model for NLP applications,” in Proc. 2003 Human Language Technology Conf. of the North American Chapter of the ACL (NAACL-HLT), 2003, pp. 134–141.

[22] E. Soper, S. Fujimoto, and Y.-Y. Yu, “BART for post-correction of OCR newspaper text,” in Proc. 7th Workshop on Noisy User-generated Text (W-NUT), 2021, pp. 284–290. doi: 10.18653/v1/2021.wnut-1.31.

[23] L. Lyu, M. Koutraki, M. Krickl, and B. Fetahu, “Neural OCR post-hoc correction of historical corpora,” Transactions of the Association for Computational Linguistics, vol. 9, pp. 479–493, 2021. doi: 10.1162/tacl_a_00379.

[24] L. Xue, N. Constant, A. Roberts, M. Kale, R. Al-Rfou, A. Siddhant, A. Barua, and C. Raffel, “mT5: A massively multilingual pre-trained text-to-text transformer,” in Proc. 2021 Conf. of the North American Chapter of the ACL:

Human Language Technologies (NAACL-HLT), 2021, pp. 483–498. doi: 10.18653/v1/2021.naacl-main.41.

[25] L. Xue, A. Barua, N. Constant, R. Al-Rfou, S. Narang, M. Kale, A. Roberts, and C. Raffel, “ByT5: Towards a token-free future with pre-trained byte-to-byte models,” Transactions of the Association for Computational Linguistics, vol. 10, pp. 291–306, 2022. doi: 10.1162/tacl_a_00461.

[26] S. Guan and D. Greene, “Advancing post-OCR correction: A comparative study of synthetic data,” in Findings of the Association for Computational Linguistics: ACL 2024, Bangkok, Thailand, Aug. 2024, pp. 6036–6047. doi: 10.18653/v1/2024.findings-acl.361.

[27] T. T. H. Nguyen, A. Jatowt, N.-V. Nguyen, M. Coustaty, and A. Doucet, “Neural machine translation with BERT for post-OCR error detection and correction,” in Proc. ACM/IEEE Joint Conf. on Digital Libraries (JCDL), 2020, pp. 333–336. doi: 10.1145/3383583.3398605.

[28] A. Thomas, R. Gaizauskas, and H. Lu, “Leveraging LLMs for post-OCR correction of historical newspapers,” in Proc. 3rd Workshop on Language Technologies for Historical and Ancient Languages (LT4HALA) @ LREC-COLING 2024, 2024, pp. 116–121.

[29] J. Kanerva, C. Ledins, S. Käpyaho, and F. Ginter, “OCR error post-correction with LLMs in historical documents: No free lunches,” in Proc. 3rd Workshop on Resources and Representations for Under-Resourced Languages and Domains (RESOURCEFUL-2025), 2025, pp. 38–47. arXiv:2502.01205.

[30] A. Krishna, B. P. Majumder, R. Bhat, and P. Goyal, “Upcycle your OCR: Reusing OCRs for post-OCR text correction in Romanised Sanskrit,” in Proc. 22nd Conf. on Computational Natural Language Learning (CoNLL), Brussels, Belgium, 2018, pp. 345–355. doi: 10.18653/v1/K18-1034.

[31] A. Maheshwari, N. Singh, A. Krishna, and G. Ramakrishnan, “A benchmark and dataset for post-OCR text correction in Sanskrit,” in Findings of the Association for Computational Linguistics: EMNLP 2022, Abu Dhabi, UAE, 2022, pp. 6258–6265. doi: 10.18653/v1/2022.findings-emnlp.466.

[32] H. Kashid and P. Bhattacharyya, “RoundTripOCR: A data generation technique for enhancing post-OCR error correction in low-resource Devanagari languages,” in Proc. 21st Int. Conf. on Natural Language Processing (ICON), 2024, pp. 274–284. arXiv:2412.15248.

[33] A. P. Singh, “Can OCR-VLMs read Devanagari? A stress-test benchmark and post-correction study,” arXiv:2606.29213, 2026.

[34] V. Dhananjaya, P. Demotte, S. Ranathunga, and S. Jayasena, “BERTifying Sinhala — A comprehensive analysis of pre-trained language models for Sinhala text classification,” in Proc. 13th Language Resources and Evaluation Conf. (LREC), Marseille, France, 2022, pp. 7377–7385. arXiv:2208.07864.

[35] O. Petrova, K. Bulatov, V. V. Arlazarov, and V. L. Arlazarov, “Weighted combination of per-frame recognition results for text recognition in a video stream,” Computer Optics, vol. 45, no. 1, pp. 77–89, 2021. doi: 10.18287/2412-6179-CO-795.

[36] A. Kim, C. Pethe, N. Inoue, and S. Skiena, “Cleaning dirty books: Post-OCR processing for previously scanned texts,” in Findings of the Association for Computational Linguistics: EMNLP 2021, 2021, pp. 4217–4226. doi: 10.18653/v1/2021.findings-emnlp.356.

[37] Department of Census and Statistics, Computer Literacy Statistics 2025, First Six Months Bulletin. Colombo, Sri Lanka: Department of Census and Statistics, 2025.

[38] Department of Census and Statistics, Census of Population and Housing 2012. Colombo, Sri Lanka: Department of Census and Statistics, 2012.

[39] Sri Lanka National Blindness, Visual Impairment and Disability Survey 2014–15. Colombo, Sri Lanka: Ministry of Health, 2016.

APPENDIX A: RELEASED ARTEFACTS AND REPRODUCTION

The artefacts below are what a reader needs in order to reproduce the figures reported in Chapter 6. They are referenced throughout Chapter 3 and Section 5.4.

Artefact	What it fixes
split_manifest.json	The page lists of the train, development and test partitions, the random seed and the LOCKED flag. Any re-run that does not reproduce this manifest is not running the same experiment.
The measured confusion table	192 character confusions with frequencies, the deletion and insertion profiles, and the counts they were derived from. This is the noise model the synthetic corpus was generated from.
The trained correction model	The fine-tuned checkpoint, its configuration and its tokenizer.
tools/verify_model.py	Re-runs the checkpoint over the locked test set and prints exact-match count, character error rate and elapsed time. It produced Table 5.2.
The evaluation scripts	Character and word error rate computation, the per-category classifier and the paired bootstrap, so that the reported confidence intervals can be regenerated.
results/*.json	The stored outputs every table in Chapter 6 was read from: final results, baseline comparison, the per-category decomposition, the threshold ablation, the over-correction diagnostic, the multi-view sweep and the two end-to-end runs.
The recorded environment	The pinned library versions under which the reported numbers hold, as recorded in the configuration module and the integration contract. Section 5.1.2 explains why the pin is part of the apparatus.
The listening-study pack	Synthesised audio for ten sentences in two conditions, six counterbalanced participant folders, the response sheet, the condition key, the collected transcripts and a scoring script that reproduces Tables 6.7 to 6.14 from them.

APPENDIX B: DISCLOSURE OF GENERATIVE AI USE

This appendix discloses the use of generative artificial intelligence tools in the preparation of this dissertation, as required.

Activity	Use of generative AI
Research design, hypotheses and the win condition	No AI use. The experimental design, the decision to lock the test set and the pre-registered win condition are my own.
Corpus construction and ground-truth transcription	No AI use. I transcribed all ground truth manually.
Model training and evaluation	No AI use in the sense of generated content. The models reported are my own fine-tuned models, and every reported figure is the output of my own evaluation code run on my own data.
Software implementation	AI coding assistance was used for routine implementation and debugging. I reviewed and tested all code, and it is covered by the test suite described in Section 5.4.
Literature search	AI assistance was used to locate candidate references. I verified every citation against a publisher page, the ACL Anthology record, Crossref or arXiv, and excluded references whose details I could not confirm rather than estimating them.
Drafting and editing of this dissertation	AI assistance was used in drafting and editing prose. The structure, the arguments, the interpretation of results and all factual content are my own.
Figures and tables	Produced by my own plotting and document-building scripts from the stored result files.